arXiv:2606.03748v1 [cs.CV] 2 Jun 2026

Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models

Glenn Jocher Jing Qiu Mengyu Liu Shuai Lyu
Fatih Cagatay Akyon Muhammet Esat Kalfaoglu

Ultralytics
{glenn, jing, mengyu, louis, fatih, esat}@ultralytics.com

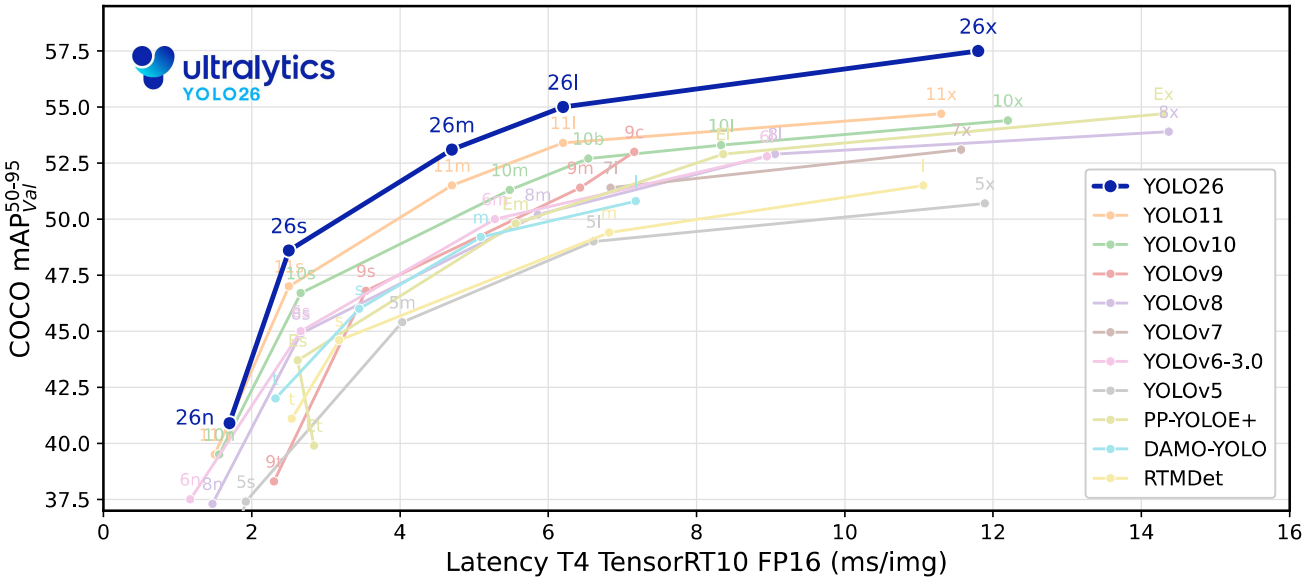


Figure 1. Accuracy–latency trade-off on COCO `val2017`. YOLO26 variants sit on or advance the Pareto front over prior YOLO versions and other real-time detectors, with the strongest AP–latency trade-off at the medium, large, and extra-large scales. Latency is measured on an NVIDIA T4 GPU with TensorRT FP16.

Abstract

Real-time vision demands models that are accurate, efficient, and simple to deploy across diverse hardware. The YOLO family has become widely deployed for this reason, yet most YOLO detectors still rely on non-maximum suppression at inference, carry heavy detection heads due to Distribution Focal Loss, require long training schedules, and can leave the smallest objects without positive label assignments. We present Ultralytics YOLO26, a unified real-time vision model family that addresses these limitations through coordinated architecture and training advances. Architecturally, YOLO26 uses a dual-head design for native NMS-free end-to-end inference and removes DFL entirely, yielding a lighter head with unconstrained regression range. For training, three techniques jointly improve accuracy while reducing cost: MuSGD, a hybrid Muon–SGD optimizer adapted from large language model training; Progressive Loss, which shifts supervision toward the inference-time head; and STAL, a label assignment strategy that guarantees positive coverage for small objects. Beyond detection, YOLO26 introduces task-specific head and loss designs for instance segmentation, pose estimation, and oriented detection, producing consistent gains across tasks and scales. The family spans five scales (n/s/m/l/x) and supports detection, instance segmentation, pose estimation, classification, and oriented detection in a single pipeline, with an open-vocabulary extension, YOLOE-26, for text-, visual-, and prompt-free inference. Across all scales, YOLO26 achieves 40.9–57.5 mAP on COCO at 1.7–11.8,ms T4 TensorRT latency, advancing the accuracy–latency Pareto front over prior real-time detectors, while

YOLOE-26x reaches 40.6 AP on LVIS minival under text prompting. Code and models are available at https://github.com/ultralytics/ultralytics.

1. Introduction

Real-time object detection is a cornerstone of practical computer vision, powering applications from autonomous driving and robotics to surveillance and augmented reality, often on edge devices with tight latency and power budgets. The field has advanced through several architectural shifts, but the central challenge remains unchanged: improving accuracy without sacrificing deployment simplicity and runtime efficiency.

Two-stage pipelines such as Faster R-CNN [60] set strong accuracy baselines but at considerable inference cost. One-stage detectors (SSD [44], RetinaNet [39], YOLOv3 [59], Ultralytics YOLOv5 [28]) traded proposal generation for dense prediction, drastically reducing latency. Anchor-free designs such as FCOS [66] and Ultralytics YOLOv8 [69] further simplified the detection head, while YOLOv10 [78] introduced consistent dual assignments to enable NMS-free inference. In parallel, DETR [5] cast detection as end-to-end set prediction, and its real-time descendants (RT-DETR [98], D-FINE [55], DEIM [21], RF-DETR [62]) have narrowed the accuracy gap with CNN-based detectors on standard benchmarks. However, these transformer-based models often depend on large pretrained vision backbones, deformable attention or other custom operators, or fixed input resolutions, which can complicate deployment across heterogeneous hardware targets and reduce portability across edge-oriented inference backends. Throughout these shifts, the YOLO family has remained the most widely deployed real-time detector in industry.

Two structural properties underpin this durability. First, *deployment universality*: YOLO models rely on standard convolutional operators, enabling native export to TensorRT [53], ONNX [54], CoreML [1], TFLite [17], OpenVINO [24], NCNN [51], and ExecuTorch [57] across cloud, mobile, and embedded platforms; Sec. 3.5 summarizes the broader export surface [71]. Second, *multi-task unification*: a common backbone and neck support object detection, instance segmentation, image classification, pose estimation, and oriented bounding box (OBB) detection within a single training and deployment stack [73]. These properties make the YOLO paradigm a strong foundation for continued progress, provided its remaining limitations are addressed directly.

Despite this strong position, several concrete limitations persist across current YOLO-family detectors. **(a) NMS dependency and suboptimal dual-head training.** Most CNN-based detectors still rely on non-maximum suppression at inference. YOLOv10 [78] introduced a dual-head design to remove NMS, but applies identical loss weights to both heads throughout training: the one-to-one branch, the only head used at inference, receives the same optimization pressure as the dense one-to-many branch, leaving it under-trained relative to what targeted supervision could achieve. **(b) DFL parameter bloat and range limitation.** The Distribution Focal Loss (DFL) module adopted since YOLOv8 [69] expands bounding-box regression from 4 scalars to $4K$ logits per spatial location (typically $K{=}16$), inflating head parameters, an especially unfavorable trade-off for nano-scale models where the head can dominate total parameter count. For instance, YOLO11n with DFL has 2.6M parameters and 6.5 GFLOPs, whereas removing DFL reduces these figures to only 2.3M parameters and 5.2 GFLOPs, a 12% parameter and 20% FLOP reduction from this single module alone. DFL also imposes a finite regression range of $(K{-}1) \times$ stride pixels per side, which becomes restrictive for large objects at high resolution. D-FINE [55] addresses this by increasing K, but the additional bins further compound head cost. **(c) Long training schedules.** Standard SGD recipes still require roughly 600 epochs to reach competitive COCO accuracy, making rapid iteration expensive. Recent work on the Muon optimizer [40] has demonstrated roughly $2\times$ computational efficiency over AdamW in large language model training, yet no prior work has adapted Muon to object detection. **(d) TAL zero-assignment for small objects.** The Task-Aligned Learning (TAL) [14] label-assignment strategy selects candidate anchors based on geometric containment within ground-truth boxes. After downsampling, objects whose spatial extent falls below the minimum stride have *no* anchor inside their box, receiving zero positive assignments and zero gradient signal. In this regime, the smallest objects can receive no positive candidates and contribute no localization or classification signal during training.

We present **Ultralytics YOLO26**, a unified real-time vision model family built on YOLO11 [70]. At the detector level, YOLO26 adopts a dual-head design for native NMS-free inference and removes DFL entirely, yielding a lighter regression head with unconstrained range. To recover localization quality and improve optimization, YOLO26 combines three complementary training components: *MuSGD*, a hybrid Muon–SGD optimizer [40]; *Progressive Loss*, which gradually shifts supervision toward the inference-time one-to-one head; and *STAL* (Small-Target-Aware Label Assignment), which guarantees positive candidate coverage for tiny objects under TAL.

The resulting family spans five size variants (n/s/m/l/x) and supports detection, instance segmentation, classification, pose estimation, and OBB detection. Beyond the shared detector, YOLO26 introduces task-specific extensions for instance segmentation, pose estimation, and oriented detection through a multi-scale proto pathway with

auxiliary semantic supervision, an RLE-based uncertainty-aware keypoint objective [33], and a revised long-edge angle formulation with dedicated angle supervision. Across scales, the YOLO26 family combines these extensions with the shared detector improvements, improving over YOLO11 by up to +3.7 mask AP on COCO instance segmentation, +7.2 pose AP on COCO keypoints, and +3.4 mAP on DOTA-v1.0 OBB detection.

Beyond closed-set detection, we extend the YOLO26 family to open-vocabulary scenarios by building **YOLOE-26**, which instantiates the open-vocabulary formulation of YOLOE [79] on the YOLO26 detector. YOLOE-26 retains the three inference modes of YOLOE (text-prompted, visual-prompted, and prompt-free) while introducing a stronger detector backbone, an upgraded text encoder (MobileCLIP2 [13]), a pseudo-label data engine, and decoupled segmentation training. On LVIS minival, YOLOE-26x reaches 40.6 AP under text prompting, surpassing DetCLIP-T [91] by +6.2 AP (see Sec. S7).

Figure 1 summarizes the accuracy–latency trade-off on COCO `val2017`. Across all model scales, YOLO26 sits on or advances the Pareto front, with the strongest AP–latency trade-off at the medium, large, and extra-large scales. At matched scales, YOLO26 improves COCO AP over YOLO11 by 1.6–2.8 points (full per-scale comparison in Sec. S4) while also outperforming other recent real-time detectors across the accuracy–latency frontier.

In summary, the main contributions of this work are:

1. A **DFL-free dual-head architecture** that provides native NMS-free end-to-end inference with a lighter regression head and unconstrained bounding-box range, while retaining an optional dense branch for accuracy-critical deployment.
2. A **coordinated training pipeline** of three complementary components, applied jointly (MuSGD, Progressive Loss, and STAL), that accelerate convergence, align optimization with the end-to-end inference path, and guarantee supervision for the smallest objects.
3. **Task-specific head and loss designs** for instance segmentation (multi-scale prototype fusion with auxiliary semantic supervision), pose estimation (uncertainty-aware RLE keypoint regression), and oriented detection (long-edge angle formulation with aspect-ratio-aware supervision), each yielding consistent gains over YOLO11 within a single unified family.
4. **YOLOE-26**, an open-vocabulary extension that applies the YOLOE [79] formulation to the YOLO26 detector with an upgraded text encoder, a pseudo-label data engine, and decoupled segmentation training, reaching 40.6 AP on LVIS minival under text prompting.

2. Related Work

2.1. CNN-based Object Detection: From Two-Stage to NMS-Free

CNN-based object detection has progressed through design shifts that balance accuracy and efficiency. Early approaches were dominated by *two-stage* pipelines that first generate candidate regions and then classify and refine them. R-CNN introduced this paradigm by combining region proposals with CNN features, Fast R-CNN improved efficiency by sharing computation across proposals via RoI pooling, and Faster R-CNN integrated proposal generation into the network with an RPN to enable end-to-end training and a strong speed–accuracy trade-off [15, 16, 60].

To reduce latency, *one-stage* detectors remove the proposal stage and predict classes and boxes densely over feature maps. SSD demonstrated efficient multi-scale dense prediction with default boxes [44], while RetinaNet addressed foreground–background imbalance via Focal Loss to improve dense detection accuracy [39]. The YOLO line targets real-time operation, with YOLOv3 strengthening multi-scale prediction and backbone design [59], and Ultralytics YOLOv5 providing a widely adopted, deployment-oriented implementation with refined training recipes [28].

Anchor-free detectors further simplify dense prediction by removing hand-designed anchors and improving portability across datasets. FCOS formulates detection as per-pixel classification and box regression on feature maps [66], while CenterNet represents objects via keypoints and their geometric relations [12]. Recent real-time systems have also adopted anchor-free heads in practice; Ultralytics YOLOv8 follows this direction with an anchor-free split-head design and deployment-oriented refinements for accuracy and efficiency [69].

Beyond anchors, recent work targets *end-to-end* detection by reducing reliance on post-processing, especially non-maximum suppression (NMS). YOLOv10 enables NMS-free inference via consistent dual assignments with two training branches: a one-to-many branch for dense supervision and a one-to-one branch that learns a single matched prediction per ground-truth instance for direct decoding at inference [78].

2.2. Transformer-based Object Detection

Transformer-based detectors cast detection as end-to-end *set prediction* with one-to-one Hungarian matching, removing anchors and NMS. DETR introduced this formulation with a transformer encoder–decoder and a fixed set of object queries [5]. Deformable DETR improved efficiency and convergence via sparse deformable attention, and introduced iterative box refinement and a two-stage top-K proposal initialization variant [100]. Follow-up work refined query design and training: DAB-DETR parameter-

izes queries as learnable anchor boxes refined across decoder layers [42], DN-DETR accelerates convergence with denoising queries built from noisy ground-truth targets [29], and DINO further improves initialization and optimization using mixed query selection, contrastive denoising, and a look-forward-twice refinement scheme [97]. Beyond pure one-to-one training, hybrid matching methods increase positive supervision by combining one-to-one with auxiliary one-to-many assignments during training (e.g., H-DETR) while preserving one-to-one inference, and Group DETR similarly provides richer supervision via multiple query groups with per-group matching [6, 25].

Transformer detectors have also been adapted for real-time use by improving multi-scale feature handling and reducing computational overhead. RT-DETR targets practical speed–accuracy trade-offs via an efficient encoder design and deployment-oriented choices [98], with later refinements such as RT-DETRv4 exploring stronger distillation strategies for compact models [37]. Concurrently, methods such as D-FINE and DEIM improve localization and training efficacy through refined regression/optimization and matching-aware objectives, with follow-up versions (e.g., DEIMv2) continuing to optimize convergence and accuracy in real-time DETR pipelines [20, 21, 55]. Lightweight designs such as LW-DETR further streamline the architecture for low-latency deployment [7], and RF-DETR represents complementary efforts to build strong real-time detection transformer families with practical accuracy–latency trade-offs [62].

2.3. Instance Segmentation

Instance segmentation extends object detection by predicting a pixel-accurate mask for each instance, with a persistent trade-off between accuracy and real-time efficiency. Mask R-CNN augments a two-stage detector with an RoI-aligned mask branch to produce high-quality masks [19]. To avoid per-RoI computation, proposal-free methods predict masks fully convolutionally: CondInst generates instance-conditioned mask heads via dynamically predicted convolution parameters applied to shared mask features [67], while SOLOv2 assigns instances to locations and predicts dynamic mask kernels per location to produce masks from shared mask features [82].

For real-time settings, *prototype-based* approaches construct instance masks from a small set of shared bases. YOLACT predicts global prototype masks and per-instance coefficients, forming each mask through a linear combination followed by cropping, which enables fast mask generation with competitive accuracy [2]. Ultralytics YOLO segmentation heads adopt a closely related prototype–coefficient formulation (e.g., YOLO11), predicting shared prototypes together with per-instance mask coefficients from detection features to build masks with minimal per-instance overhead [70].

More recently, transformer-based instance segmentation predicts masks via set-based decoding with stronger global context. Mask2Former employs masked-attention decoding to output a set of instance masks and corresponding classes [8]. MaskDINO further improves convergence and mask quality by incorporating denoising-based training and enhanced mask decoding within a unified transformer framework [31].

2.4. Pose Estimation

Human pose estimation progressed from direct coordinate regression (DeepPose [68]) to heatmap prediction via encoder–decoder networks (Stacked Hourglass [52]) and deconvolutional baselines (SimpleBaselines [85]). HRNet maintains high-resolution representations through parallel subnetwork fusion [64], while OpenPose takes a bottom-up approach by jointly predicting keypoints and Part Affinity Fields [4]. ViTPose demonstrated scalable transformer backbones for this task [87], and RTMPose achieves real-time performance via CSPNeXt with a coordinate classification head [27].

Within YOLO-based pose estimation, YOLO-Pose introduced a heatmap-free single-stage formulation that jointly detects persons and regresses keypoints using a scale-aware OKS-based training objective [48]. Later YOLOv7 integrated a keypoint head into the broader YOLO family, and YOLOv8 and YOLO11 continued this direction with anchor-free pose heads [69, 70, 80]. YOLO26 builds on this direct-regression lineage and further augments it with RLE [33], replacing conventional regression with a normalizing-flow-based probabilistic objective for more accurate and calibrated keypoint localization.

2.5. Oriented Bounding Box Detection

Oriented object detection extends axis-aligned detection to boxes at arbitrary rotation angles, motivated by aerial imagery and scene text where upright boxes produce excessive overlap. Early methods adapted region-proposal networks with rotated anchors [47], while two-stage detectors such as RoI Transformer [10] and Oriented R-CNN [86] applied rotation-aware heads to achieve strong accuracy. A central difficulty is *angle representation*: naïve regression suffers from boundary discontinuities at the periodicity boundary ($\pm\pi/2$ for symmetric boxes), where a small geometric change causes a large loss spike. Circular Smooth Label (CSL) [88] addresses this by recasting angle prediction as circular classification. Gaussian-based methods bypass the issue by modelling rotated boxes as 2D Gaussians and measuring similarity via Wasserstein distance (GWD [89]) or a Kalman-filter-inspired IoU surrogate (KFIoU [90]). However, the Gaussian modeling method would cause angle ambiguity of square rotated objects, and an additional angle

loss [93] was proposed to penalize angle deviation. PSC and PSCD [94] introduce phase based orientation angle coders, where PSC solves the boundary discontinuity issue and PSCD further reduces the ambiguity of square-like objects by introducing a dual-frequency phase representation. Anchor-free designs such as S2ANet [18] align features to oriented proposals for finer localization, and modern backbones such as LSKNet [36] exploit large selective kernels suited to the elongated structures common in aerial scenes.

Within the YOLO family, YOLOv8 [69] and YOLO11 [70] introduced OBB heads with a dedicated angle branch trained via ProbIoU [45]. YOLO26 extends this lineage with a dedicated aspect-ratio-aware angle loss and an optimized decoder, addressing angle ambiguity and boundary discontinuities in rotated box representations.

2.6. Open-Vocabulary Detection and Segmentation

Open-vocabulary detection methods can be broadly categorized by how they specify target categories at inference time: through text prompts, visual prompts, or no prompts at all. **Text-prompted detection** grounds natural-language queries to visual regions via vision–language pretraining. GLIP [34] reformulates detection as phrase grounding, aligning region features with token-level text embeddings through a contrastive objective. Grounding DINO [43] extends this idea to a transformer detector with tight cross-modal fusion, achieving strong zero-shot localization. YOLO-World [9] brings text-conditioned detection into the real-time regime by injecting CLIP text embeddings into a YOLO neck via re-parameterizable cross-attention, and YOLO-UniOW [41] further unifies open-world detection under a single YOLO-family model. Despite their generality, text prompts are ill-suited for objects that resist concise linguistic description, such as novel industrial defects or fine-grained biological specimens. **Visual-prompted detection** addresses this gap by supplying reference images or regions in place of text. OWL-ViT [49] and OV-DETR [95] process image exemplars through a shared CLIP encoder alongside text queries, supporting both modalities under a unified architecture. DINOv [32] explores reference regions as in-context examples for generic and referring vision tasks, while T-Rex2 [26] achieves tighter multi-modal alignment via region-level contrastive training. For segmentation, SEEM [101] and Semantic-SAM [30] handle diverse prompt types—including points, boxes, and reference masks—across panoptic and part-level granularities. **Prompt-free detection** removes the dependence on explicit queries altogether by coupling detectors with generative language models. GRiT [83] attaches an autoregressive text decoder to a region-proposal backbone for joint dense captioning and detection. DetCLIPv3 [92] trains an object captioner on web-scale data to generate rich label information for arbitrary regions, and GenerateU [38] uses a language model to produce object names in free form, decoupling recognition from any predefined vocabulary. YOLOE [79] unifies all three paradigms—text-prompted, visual-prompted, and prompt-free inference—within a single real-time model through its RepRTA, SAVPE, and LRPCHead components. Our YOLOE-26 inherits this unified formulation and advances it with a stronger backbone, an upgraded text encoder, a pseudo-label data engine, and decoupled segmentation training.

3. Methodology

YOLO26 builds on the YOLO11 family as a unified real-time vision framework, while revising both the shared detector design and the task-specific heads. At the detection level, the methodology combines a dual-head end-to-end formulation, DFL-free box regression, MuSGD, Progressive Loss, and Small-Target-Aware Label Assignment (STAL). This section first presents the shared architecture and training pipeline, and then describes the task-specific extensions for segmentation, pose estimation, oriented bounding boxes, and the open-vocabulary YOLOE-26 variant.

3.1. Overview

Relative to YOLO11 [70], YOLO26 is organized around three design goals: *end-to-end simplicity*, *deployment efficiency*, and *stronger optimization*. These goals are realized through a native NMS-free one-to-one inference path, a lightweight DFL-free regression head, and a training pipeline that couples MuSGD, Progressive Loss, and STAL.

Figure 2 summarizes how these components interact during training: the shared backbone and neck feed the one-to-many and one-to-one heads, STAL preserves tiny-object assignments, Progressive Loss reweights the branch objectives over time, and MuSGD updates the model parameters.

3.2. Architecture Design

The main paper focuses on the architectural ideas most directly tied to the detection objective, while the full model schematic and the constituent block compositions are provided in Sec. S2 of the supplementary materials; see also Supplementary Figs. S1 and S2. Table 1 maps the main detection components to their implementation names in the Ultralytics repository.

3.2.1. End-to-End NMS-Free Detection

Most CNN-based detectors rely on non-maximum suppression (NMS) to remove duplicate predictions at inference time. YOLO26 instead exposes a *dual-head* design that supports both end-to-end NMS-free decoding and conventional dense prediction, enabling deployment-specific trade-offs.

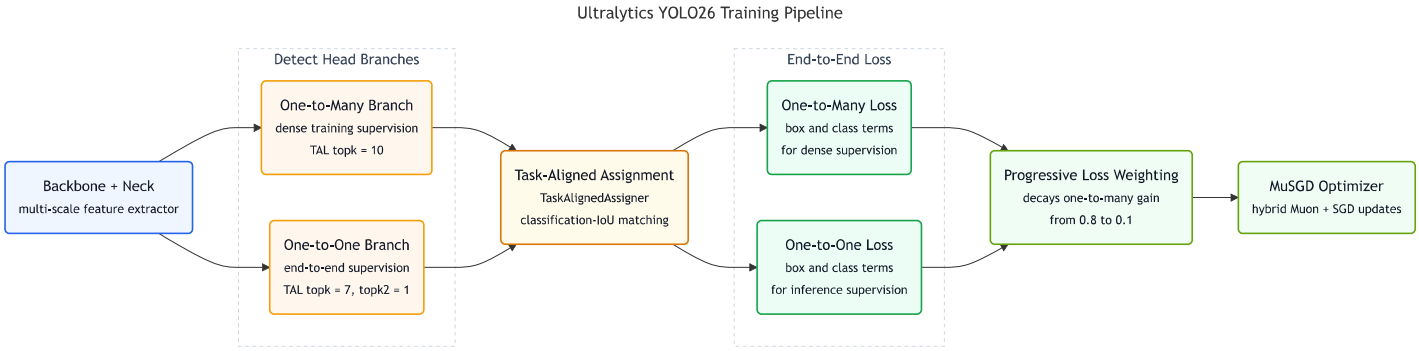


Figure 2. Training pipeline of Ultralytics YOLO26. The shared backbone and neck feed the dual detection heads, STAL improves assignment robustness for tiny targets, Progressive Loss reweights the one-to-many and one-to-one objectives over training, and MuSGD performs the final parameter updates.

Component	Implementation	Role
Dual head	`Detect`	E2E and NMS paths
Direct regression	`reg_max=1`	DFL-free boxes
STAL	`TaskAlignedAssigner`	Tiny-object coverage
Progressive Loss	`E2ELoss`	Branch reweighting
Optimizer	`MuSGD`	Faster convergence

Table 1. Main YOLO26 detection components and their corresponding implementation names in the Ultralytics repository.

One-to-One Head (default). The one-to-one head produces a fixed-size set of predictions without requiring NMS, outputting at most 300 detections per image (shape $(N, 300, 6)$ for batch size N). Following YOLOv10 [78], we train the dual heads with *consistent dual-path label assignment* based on Task-Aligned Learning (TAL) [14]: both heads use the same TAL formulation, but with different matching cardinalities. In the current implementation, the one-to-one path first forms a TAL candidate set with topk $=$ 7 and then applies a secondary topk2 $=$ 1 filter, yielding a unique end-to-end assignment per ground-truth instance and enabling direct decoding at inference.

One-to-Many Head. The one-to-many head retains the standard dense YOLO-style prediction and uses TAL with topk $=$ 10 to provide richer positive supervision during training, producing outputs of shape $(N, n_c + 4, 8400)$ where n_c is the number of classes. This head uses NMS at inference and typically yields slightly higher accuracy at the cost of additional post-processing overhead.

Overall, the dual-head design offers a practical accuracy–latency knob: the one-to-one head prioritizes simplicity and speed (NMS-free, fixed output), while the one-to-many head targets maximum accuracy when NMS overhead is acceptable.

3.2.2. Distribution Focal Loss Removal

YOLO26 removes the Distribution Focal Loss (DFL) module from the detection head. DFL was introduced in Generalized Focal Loss (GFL) [35] as a distribution-based box-regression formulation and is now widely used in recent YOLO-style detectors, including Ultralytics YOLOv8 [69] and later variants such as YOLOv9 [81], YOLOv10 [78], and YOLO11 [70]. In DFL, each box side is predicted as a discrete distribution over K bins (typically $K{=}16$) and decoded by expectation:

$$d = \sum_{i=0}^{K-1} i \cdot \mathrm{softmax}(z)_i, \quad z \in \mathbb{R}^K, \tag{1}$$

which expands regression from 4 scalars to $4K$ logits per location and increases head parameters/compute accordingly—a particularly unfavorable trade-off for nano-scale models.

DFL also imposes a finite discrete support range. Since $d \in [0, K{-}1]$ before multiplying by stride s, the maximum per-side distance is $(K{-}1)s$ pixels (thus width/height are bounded by $\approx 2(K{-}1)s$ at a given feature level). With $K{=}16$, this is $\approx 30s$ (e.g., ~ 960 pixels at $s{=}32$), which can become restrictive for large objects at high resolution (e.g., 1280). Increasing K (e.g., D-FINE uses larger bin counts) alleviates this but further increases head cost [55]. YOLO26 instead adopts a simpler regression head and recovers localization quality via complementary training objectives, namely Progressive Loss (Sec. 3.3.2) and STAL (Sec. 3.3.3). The quantitative and qualitative evidence for this design choice is provided in the DFL-removal ablation in Sec. 4.3.1. In particular, Fig. 4 uses 1280-resolution training setups to expose a failure mode that is less apparent at 640, showing that the DFL-free head more reliably preserves the full extent of large objects.

Finally, removing DFL simplifies export and improves

compatibility with constrained runtimes and accelerators that favor standard operators and minimal decoding overhead. Overall, this choice reflects an explicit accuracy–efficiency trade-off for resource-constrained deployment while preserving strong detection performance through compensating training enhancements.

3.3. Training Methodology

Starting from the shared backbone and neck features, YOLO26 optimizes the one-to-many and one-to-one heads jointly, applies STAL during label assignment to preserve supervision for tiny targets, and combines the two branch losses through Progressive Loss before updating model parameters with MuSGD. This view highlights how the proposed training components interact as a single pipeline rather than as isolated modifications.

3.3.1. MuSGD Optimizer

YOLO26 adopts *MuSGD*, a hybrid optimizer that combines Muon [40] with standard SGD-momentum [58], motivated by recent large-scale training practice [50]. Muon applies momentum updates followed by a lightweight orthogonalization of the momentum-derived update (via a few Newton–Schulz iterations), which improves update conditioning and can stabilize optimization [40]. We leverage this property for detector training while retaining SGD as a robust baseline component.

Concretely, MuSGD applies a weighted mixture of the Muon update and the SGD update to multi-dimensional parameters (e.g., convolution kernels and linear weights), and uses pure SGD for 1D parameters such as biases and normalization scales. This parameter-type split keeps scale/shift parameters stable while benefiting from orthogonalized updates on high-dimensional weight tensors, improving training stability and accelerating convergence in practice (see Sec. 4.3.2).

3.3.2. Progressive Loss

Dual-head end-to-end detectors exhibit an inherent optimization asymmetry during training. The dense one-to-many branch receives broader positive supervision and is therefore easier to optimize in the early stage, whereas the one-to-one branch is more constrained but ultimately determines the model's end-to-end inference behavior. Applying fixed, identical loss weights to both branches throughout training under-utilizes this asymmetry and can leave the inference branch insufficiently optimized.

To address this issue, YOLO26 introduces *Progressive Loss*, a curriculum-style reweighting strategy that gradually transfers optimization emphasis from the dense branch to the one-to-one branch over the course of training. Early in optimization, the dense branch is emphasized to stabilize feature learning and provide reliable supervision. As training progresses, the one-to-one branch receives increasingly greater emphasis so that the optimization objective becomes better aligned with the NMS-free inference path used at deployment.

Formally, the total detection loss is written as

$$\mathcal{L}_{\text{total}} = \alpha(t)\,\mathcal{L}_{\text{one2many}} + \big(1-\alpha(t)\big)\,\mathcal{L}_{\text{one2one}}, \tag{2}$$

where t denotes the current epoch and $\alpha(t)$ is a linearly decreasing schedule defined by

$$\alpha(t) = \max\left(1 - \frac{t}{\max(E-1,\,1)},\, 0\right)\,(\alpha_{\text{init}} - \alpha_{\text{final}}) + \alpha_{\text{final}}, \tag{3}$$

with E the total number of training epochs, α_{init} the initial one-to-many weight, and α_{final} the final one-to-many weight. Thus, the objective transitions smoothly from dense supervision to inference-oriented supervision over time. The exact values of α_{init}, α_{final}, and branch-specific assignment settings are deferred to the implementation details in Sec. 4.1.

Progressive Loss complements the asymmetric assignment used by the two branches: the one-to-many branch benefits from richer candidate supervision, while the one-to-one branch uses a stricter assignment tailored to end-to-end prediction. By matching the loss emphasis to these distinct roles, Progressive Loss improves early-stage optimization stability while better aligning the final model with deployment-time behavior.

3.3.3. Small-Target-Aware Label Assignment (STAL)

Task-aligned assignment first restricts supervision to anchor centers that fall inside each ground-truth box. While this geometric filtering is effective for normal-scale objects, it becomes brittle for very small instances: after feature-map discretization, a tiny box may contain no valid anchor centers at all. In that case, the object receives zero positive candidates and contributes no localization or classification signal during training.

YOLO26 addresses this failure mode with *Small-Target-Aware Label Assignment* (STAL), which decouples the geometry used for candidate selection from the geometry used for regression. Let a ground-truth box be parameterized as $g_i = (x_i, y_i, w_i, h_i)$, and let $s_{\min}$ denote the smallest feature-pyramid stride. During candidate filtering only, STAL constructs an assignment surrogate

$$\tilde{g}_i = (x_i, y_i, \tilde{w}_i, \tilde{h}_i), \tag{4}$$

where each spatial dimension is adjusted independently as

$$\tilde{d}_i = \begin{cases} s_{\text{ref}}, & d_i < s_{\min}, \\ d_i, & \text{otherwise}, \end{cases} \qquad d_i \in \{w_i, h_i\}, \tag{5}$$

where s_{ref} is a fixed reference size derived from the feature pyramid. In the current implementation, it is set to the second pyramid stride when available; the exact default values

are given in Sec. 4.1. For each ground-truth object i and anchor center a_j, we then define a binary candidate mask

$$M_{ij} = \begin{cases} 1, & \text{if anchor center } a_j \text{ lies inside } \tilde{g}_i, \\ 0, & \text{otherwise,} \end{cases} \tag{6}$$

where a_j is the j-th anchor center.

Importantly, STAL modifies only the candidate-selection mask. The original box g_i is preserved for task-aligned scoring, final target assignment, and box regression, so the detector is still optimized against the true object extent. This makes STAL deliberately conservative: it does not alter localization targets or inflate supervision for ordinary objects, but it prevents pathological zero-positive cases for tiny instances that would otherwise be dropped by the assignment pipeline.

3.4. Task-Specific Extensions

Beyond the shared detection architecture and training methodology, YOLO26 introduces task-specific extensions for instance segmentation, pose estimation, and oriented bounding box detection. These extensions preserve the common end-to-end backbone and neck while adapting the prediction heads and supervision schemes to the structural requirements of each task.

Image Classification. YOLO26 classification variants reuse the standard Ultralytics `Classify` head on the shared backbone. Because the classification branch introduces no new task-specific decoding rule, it is treated as a supported model-family task rather than a separate methodological contribution; the optimizer transfer check is reported in Sec. S1 of the supplementary materials.

3.4.1. Instance Segmentation

YOLO-style instance segmentation adopts a prototype-based representation, where a shared prototype tensor is predicted once per image and each positive instance predicts a coefficient vector to reconstruct its mask [2, 70]. Given a prototype tensor $\mathbf{P} \in \mathbb{R}^{K\times H\times W}$, where $\mathbf{P}_k \in \mathbb{R}^{H\times W}$ denotes the k-th prototype map, and instance-specific coefficients $\mathbf{c}_i \in \mathbb{R}^K$, the mask logit for instance i, denoted by $\hat{\mathbf{M}}_i \in \mathbb{R}^{H\times W}$, is formed as

$$\hat{\mathbf{M}}_i = \sum_{k=1}^{K} c_{ik}\mathbf{P}_k. \tag{7}$$

YOLO26 retains this lightweight reconstruction rule, while introducing two segmentation-specific integrations: a multi-scale proto pathway for prototype generation and a training-only auxiliary semantic supervision branch.

Multi-Scale Proto Module. Let $\{\mathbf{X}_\ell\}_{\ell=1}^{L}$ denote the segmentation features across pyramid levels, with $\mathbf{X}_1$ being the highest-resolution feature. Standard YOLO segmentation heads generate prototypes from $\mathbf{X}_1$ alone. YOLO26 instead constructs a fused proto feature

$$\mathbf{F}_{\text{proto}} = \mathbf{X}_1 + \sum_{\ell=2}^{L} \mathcal{U}(\phi_\ell(\mathbf{X}_\ell)), \tag{8}$$

where $\phi_\ell(\cdot)$ is a learnable 1×1 projection and $\mathcal{U}(\cdot)$ upsamples features to the spatial resolution of $\mathbf{X}_1$. The shared prototype tensor is then produced as

$$\mathbf{P} = \tilde{\mathcal{G}}(\mathbf{F}_{\text{proto}}), \tag{9}$$

where $\tilde{\mathcal{G}}(\cdot)$ denotes the prototype-generation stack applied to the fused feature. This modification preserves the prototype-coefficient formulation while enriching the prototypes with higher-level semantic context and broader scale coverage.

Auxiliary Semantic Segmentation Loss. YOLO26 further attaches a training-only semantic segmentation branch to the shared fused feature $\mathbf{F}_{\text{proto}}$, predicting dense per-class logits before prototype generation. The supervision target is a semantic mask derived from the instance annotations by merging mask pixels according to their class labels. We supervise this branch with a balanced BCE+Dice objective, which provides dense class-aware gradients in addition to the instance-mask loss. Importantly, this branch is auxiliary: it is inactive at evaluation time and is explicitly removed during model fusion, so it introduces no additional inference-time cost.

3.4.2. Pose Estimation

In prior YOLO pose models, a pose head is attached to directly regress the keypoint coordinates (x, y) and visibility scores, and training uses an Object Keypoint Similarity (OKS)-based loss [48], which normalizes keypoint localization error by person scale and per-keypoint OKS constants. YOLO26 extends this scheme with *Residual Log-Likelihood Estimation* (RLE) [33] to achieve principled per-joint uncertainty modeling. In addition to coordinate outputs, a parallel *sigma branch* predicts per-axis uncertainty $\boldsymbol{\sigma}=(\sigma_x, \sigma_y) \in (0, 1)^2$ for each joint. The coordinate residual is normalized accordingly:

$$\boldsymbol{\varepsilon} = \frac{\hat{\mathbf{x}} - \mathbf{x}^*}{\boldsymbol{\sigma}}, \tag{10}$$

where $\hat{\mathbf{x}}$ is the predicted joint location and $\mathbf{x}^*$ is the ground truth. A shared *RealNVP* normalizing flow [11] estimates $\log \varphi(\boldsymbol{\varepsilon})$, the log-density of the normalized residual under a learned distribution. The training objective combines an

explicit base (e.g., Laplace) negative log-likelihood term $\mathcal{L}_q$ with the learned residual term:

$$\begin{aligned}\mathcal{L}_{\text{RLE}} &= \log \boldsymbol{\sigma} - \log \varphi(\boldsymbol{\varepsilon}) + \mathcal{L}_q(\sigma, \varepsilon) \\ &= \log \boldsymbol{\sigma} - \log \varphi(\boldsymbol{\varepsilon}) + \underbrace{\log(2\boldsymbol{\sigma}) + |\boldsymbol{\varepsilon}|}_{-\log \text{ Laplace}(\hat{\mathbf{x}}; \mathbf{x}^*, \boldsymbol{\sigma})}, \end{aligned} \quad (11)$$

where the residual term anchors the uncertainty scale and stabilizes early training. Joints with higher predicted $\boldsymbol{\sigma}$—e.g., occluded or inherently ambiguous keypoints—are effectively down-weighted, yielding improved localization without discarding any predictions. The decoding path is further streamlined for inference speed.

3.4.3. Oriented Bounding Box Detection

YOLO26 OBB Parameterization. In YOLO OBB models, a separate branch is adopted to predict the orientation angle. In previous versions, the oriented bounding box follows the OpenCV [3] convention, where the angle is defined as the acute angle between the box width and the positive x-axis, with a range of $(0, 90^\circ]$. Under this definition, width and height are not strictly fixed, which introduces ambiguity for objects whose orientations are close to 0 or 90°. Small orientation changes near these boundaries can cause edge swapping between width and height, making angle regression discontinuous and unstable. In YOLO26, the angle definition is changed to the long-edge definition following MMRotate [99], where the angle range is $[-45^\circ, 135^\circ)$ and the width is constrained to be larger than the height. This formulation alleviates the boundary ambiguity near 0 or 90° and reduces the instability caused by edge swapping. In addition, previous OBB models predict an oriented angle logit z and then map it through a sigmoid transform,

$$\hat{\theta} = (\sigma(z) - 0.25)\,\pi, \quad (12)$$

which compresses predictions into a fixed interval and introduces an additional squashing nonlinearity near the interval boundaries. YOLO26 instead predicts the angle directly,

$$\hat{\theta} = z, \quad (13)$$

removing the extra squashing nonlinearity.

Angle-Loss for Square Objects. For square or near-square objects, the ProbIoU [45] loss used in YOLO11 models becomes insensitive to angle variations because the Gaussian representation is nearly invariant to rotation when width $\approx$ height, making angle prediction ambiguous and unstable. To address this issue, an angle loss is specifically designed for square objects in YOLO26. We first recap the angle supervision used in the rotated box formulation. Since an oriented rectangle is unchanged under a 180° rotation, (x, y, w, h, θ) and $(x, y, w, h, \theta + \pi)$ represent the same geometry. The angular residual should therefore be measured modulo π rather than on the real line. Let

$$\Delta\theta_i = \hat{\theta}_i - \theta_i^*, \qquad \tilde{\Delta\theta}_i = \Delta\theta_i - \text{round}\left(\frac{\Delta\theta_i}{\pi}\right)\pi, \quad (14)$$

where $\hat{\theta}_i$ and θ_i^* denote the predicted and target angles, respectively. The angle loss is then defined as

$$\begin{aligned}\mathcal{L}_{\text{angle}} &= \frac{1}{S}\sum_{i\in\mathcal{F}} q_i\,\omega_i\,\sin^2\left(2\tilde{\Delta\theta}_i\right), \\ S = \max\left(\sum_{i\in\mathcal{F}} q_i, 1\right), &\qquad \omega_i = \exp\left(-\frac{\log^2(w_i^*/h_i^*)}{\lambda^2}\right). \end{aligned} \quad (15)$$

where $\mathcal{F}$ denotes the foreground assignments, q_i is the assignment weight produced by TAL, S is the corresponding normalization factor, ω_i denotes an aspect-ratio-aware factor computed from the target box dimensions (w_i^*, h_i^*), and λ is the hyper-parameter. The double-angle penalty is used as auxiliary supervision for square and near-square boxes, for which rotations separated by 90° become geometrically ambiguous. Elongated boxes receive smaller ω_i and remain primarily constrained by the rotated IoU loss.

3.5. Model Variants and Deployment

YOLO26 provides a unified model family spanning five size variants (n, s, m, l, x) across multiple computer vision tasks: object detection, instance segmentation, image classification, pose estimation, and oriented object detection [73]. Each variant supports training, validation, inference, and native PyTorch checkpoints [72, 74, 75]. For deployment, Ultralytics supports 19 export targets beyond PyTorch: TorchScript, ONNX, OpenVINO, TensorRT, CoreML, TensorFlow SavedModel, TensorFlow GraphDef, TensorFlow Lite, TensorFlow Edge TPU, TensorFlow.js, PaddlePaddle, MNN, NCNN, IMX, RKNN, ExecuTorch, Axelera AI, DEEPX, and Qualcomm QNN [71].

Figure 3 illustrates the deployment side of the framework. After training, the same YOLO26 model can be executed through the default one-to-one NMS-free path or the optional one-to-many path with NMS, while preserving compatibility with standard export targets. Some runtimes that do not support the top-K operations required by end-to-end decoding automatically fall back to the non-end-to-end branch during export. This separation between training and deployment emphasizes that YOLO26 is designed not only for strong optimization behavior but also for practical inference integration across heterogeneous runtimes.

The architecture achieves up to 43% faster CPU inference compared to previous YOLO versions (YOLO26n vs. YOLO11n in ONNX format, benchmarked on an Intel Xeon CPU @ 2.00 GHz), with reduced model size and memory

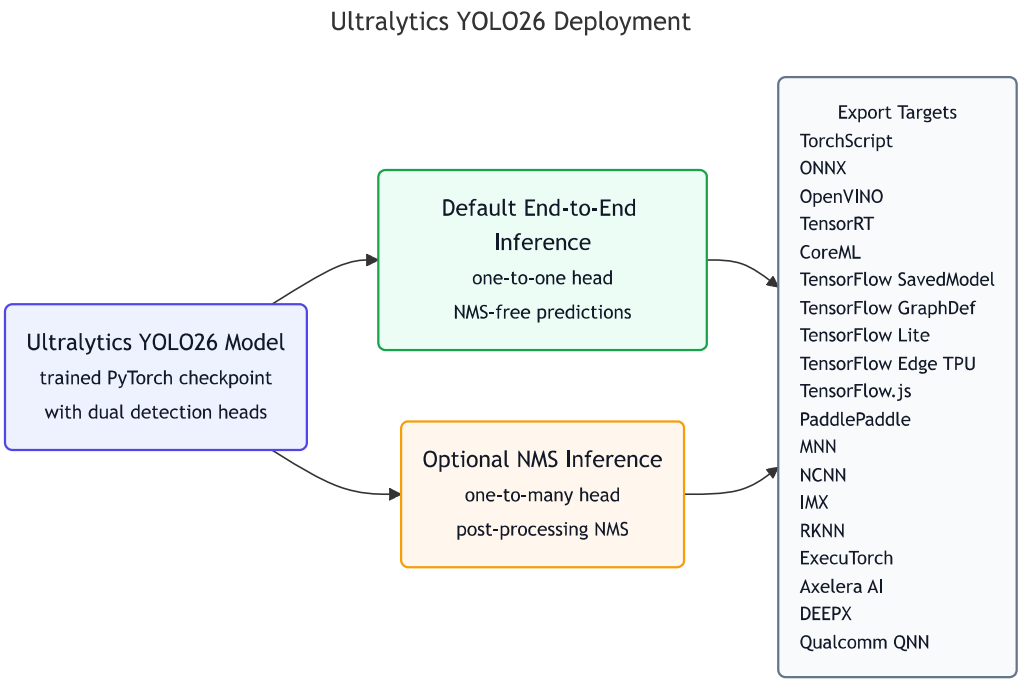


Figure 3. Deployment pipeline of Ultralytics YOLO26. A trained model supports two inference paths: the default one-to-one NMS-free path and the optional one-to-many path with NMS, while preserving compatibility with a broad set of export targets.

footprint, making it particularly suitable for edge deployment scenarios where GPU acceleration is unavailable or impractical.

3.6. YOLOE-26: Open-Vocabulary Detection and Segmentation

YOLOE [79] extends the YOLO framework with embedding-based classification to support text-prompted, visual-prompted, and prompt-free inference within a single model. For text prompts, the Re-parameterizable Region-Text Alignment (RepRTA) strategy [79] uses a MobileCLIP [77] text encoder and a lightweight auxiliary network to align region features with text embeddings. For visual prompts, the Spatial-Aware Visual Prompt Embedding module (`SAVPE`) [79] produces visual embeddings through decoupled semantic and activation branches. For prompt-free inference, the Lightweight Region Proposal and Classification head (`LRPCHead`) [79] leverages a built-in vocabulary and dedicated feature embeddings to detect generic objects without requiring a language model at inference time.

In the current implementation, YOLOE and YOLOE-26 instantiate the `BNContrastiveHead`. Let $\mathbf{F}_l$ denote the detection feature map at pyramid level l. YOLOE-26 predicts box-branch outputs and prompt-conditioned classification scores through separate heads:

$$\mathbf{B}_l = \mathcal{B}_l(\mathbf{F}_l), \qquad \mathbf{Z}_l = \mathcal{E}_l(\mathbf{F}_l), \tag{16}$$

where $\mathbf{B}_l$ denotes the box-branch outputs and $\mathbf{Z}_l$ denotes the classification embeddings. Given normalized prompt embeddings $\mathbf{W} \in \mathbb{R}^{B\times K\times C}$ for K categories, the score map is

$$\mathbf{S}_l = \exp(\tau) \cdot \big(\mathrm{BN}(\mathbf{Z}_l) \otimes \mathbf{W}\big) + b, \tag{17}$$

where $\mathbf{S}_l \in \mathbb{R}^{B\times K\times H\times W}$, which serves as the open-vocabulary equivalent of the classification logits in standard YOLO detectors, and is subsequently passed through a sigmoid function to produce the final per-class confidence scores. $\mathrm{BN}(\cdot)$ applies batch normalization to classification embeddings, $\otimes$ denotes the inner product of the channel category across spatial locations, and τ, b are learnable scalars initialized to -1 and -10, respectively. The box and classification branches remain aligned by shared spatial indices, so each location on the level l produces a paired box prediction and class-score vector. YOLOE is trained in three stages: a text-prompt stage (TP), a visual-prompt stage (VP) and a prompt-free stage (PF). The TP stage serves as the shared initialization stage, and the VP and PF models are each fine-tuned independently from the TP checkpoint.

YOLOE-26 instantiates the open-vocabulary formulation of YOLOE [79] on top of the YOLO26 detector family. It retains the same three inference modes while introducing

four modifications. **(1) Backbone upgrade.** The original YOLO11-based detector is replaced by the YOLO26 backbone, neck, and end-to-end detection stack. In practice, the TP model is initialized from released pretrained YOLO26 detector weights, transferring the closed-set design improvements described above to the open-vocabulary setting without altering the prompting interface. **(2) Text encoder upgrade.** The MobileCLIP [77] text encoder is upgraded to MobileCLIP2 [13], yielding stronger text–visual alignment. **(3) Data engine.** Upon visualizing the training set, we observed that a substantial number of objects are left unannotated in the original annotations, yet can be reliably detected by a trained YOLOE model. Motivated by this observation, we employ a pretrained YOLOE teacher, prompted with a built-in vocabulary of 4585 classes [22], to refine the training set. Predicted boxes that are absent from the original annotations, have $\mathrm{IoU} < 0.5$ with any existing ground-truth box, and exceed a confidence threshold of 0.5 are appended as pseudo-labels for a second round of training. **(4) Decoupled segmentation training.** The segmentation head is disabled during the initial text-prompt training stage and is trained in a separate subsequent stage from the TP checkpoint, allowing the backbone to focus on open-vocabulary detection. This contrasts with the original YOLOE, which trains the segmentation head jointly with the text-prompt branch.

4. Experiments

YOLO26 is evaluated on standard object detection, instance segmentation, pose estimation, oriented bounding box detection, and open-vocabulary detection. Implementation details and ablation studies are presented first, followed by the main COCO detection results, the task-specific results, and the YOLOE-26 results.

4.1. Implementation Details

Unless otherwise stated, YOLO26 detection models are trained in end-to-end mode with a direct regression head (`reg_max`=1). For the reported COCO detection benchmarks, we use a two-stage training recipe: all model sizes are first pretrained on Objects365-v1 [63] for 150 epochs, and are then fine-tuned on COCO. The COCO fine-tuning schedule is model-size dependent, using 245/70/80/60/40 epochs for the n/s/m/l/x variants, respectively. The global batch size is 128 across model scales in both stages. Exact per-size optimizer, schedule, loss, augmentation, and checkpoint-recorded internal settings for the Objects365-v1 pretraining stage and COCO fine-tuning stage are summarized in Sec. S3 of the supplementary materials and Tables S2–S7.

Across both Objects365-v1 pretraining and COCO fine-tuning, we adopt a model-size-aware augmentation policy. In both stages, larger models are regularized with stronger scale, mixup, and copy-paste augmentation overall, while YOLO26n uses the mildest recipe. Mosaic is used heavily during most of training in both stages and is disabled near the end of training through the `close_mosaic` schedule. The exact Objects365-v1 pretraining and COCO fine-tuning augmentation settings are provided in Sec. S3 of the supplementary materials and Tables S3 and S6, respectively.

The Progressive Loss weights are initialized to emphasize the one-to-many branch in the early stage and are then shifted toward the one-to-one branch over training. In the current implementation, the one-to-many and one-to-one weights are initialized as $(0.8, 0.2)$ and linearly updated over training to $(0.1, 0.9)$, respectively. The update is applied once per training epoch. We keep these values fixed across experiments unless otherwise noted, and treat them as implementation choices rather than independently tuned hyperparameters.

For STAL, the current implementation uses the default three-level detection pyramid with strides $[8, 16, 32]$. Accordingly, the smallest stride is $s_{\min} = 8$, and the reference size is set to the next stride level, $s_{\mathrm{ref}} = 16$. In practice, this means that during candidate filtering only, any ground-truth width or height below 8 pixels is clamped to 16 pixels, while the original box remains unchanged for subsequent matching and regression.

4.2. Component-Wise Ablation

Table 2 summarizes the incremental evolution from the YOLO11 baseline to the final YOLO26s configuration. Removing DFL and replacing it with direct box regression plus L1 loss largely preserves accuracy while reducing model complexity, and STAL then recovers additional performance with improved small-object assignment. We further apply a light backbone/neck refinement by inserting one additional attention layer in the detection neck, which improves accuracy while keeping latency essentially unchanged. Enabling end-to-end training with Progressive Loss then improves the one-to-one branch, while MuSGD, Objects365 pretraining, and hyperparameter search provide further gains to reach the final result.

4.3. Core Design Ablations

In this subsection, we isolate the main design choices that distinguish YOLO26 from the baseline formulation. The following ablations evaluate the effects of DFL removal, MuSGD, Progressive Loss, and STAL, showing how each component contributes to optimization behavior, accuracy, and deployment-oriented design.

4.3.1. DFL Removal

Table 2 shows that removing DFL alone costs 0.6 AP on the YOLO11s baseline (47.0 to 46.4), but this gap is fully recovered by L1 supervision (+0.2), STAL (+0.2), and the backbone/neck refinement (+0.2), yielding a strictly lighter

Model	AP (E2E)	AP (Non-E2E)	Params (M)	FLOPs (G)	Latency (ms)
YOLO11s (Baseline)	–	47.0	9.4	21.5	2.5
− DFL	–	46.4	9.1	20.1	2.3
+ L1 Loss	–	46.6	9.1	20.1	2.3
+ STAL	–	46.8	9.1	20.1	2.3
+ Backbone/neck refinement	–	47.0	9.5	20.7	2.5
+ E2E	46.4	47.0	9.5	20.7	2.5
+ Progressive Loss	46.7	47.2	9.5	20.7	2.5
+ MuSGD	47.1	47.6	9.5	20.7	2.5
+ Objects365 Pretrained	47.4	48.0	9.5	20.7	2.5
+ Hyperparameter Search	47.8	48.6	9.5	20.7	2.5

Table 2. Step-by-step ablation from the YOLO11s baseline to the final YOLO26s configuration. Replacing DFL with direct box regression and L1 loss reduces complexity with limited accuracy change, STAL and the backbone/neck refinement recover and improve the NMS-based AP, and end-to-end training with Progressive Loss improves the one-to-one branch. MuSGD, Objects365 pretraining, and hyperparameter search provide the remaining gains to the final result.

Res.	DFL	AP ↑	AP_S	AP_M	AP_L
640	w/	46.0	27.3	50.4	62.8
640	w/o	**46.3**	**27.9**	**50.6**	**63.8**
1280	w/	49.8	**36.0**	**55.7**	61.8
1280	w/o	**51.1**	35.9	55.2	**64.0**

Table 3. Controlled DFL removal ablation on COCO using YOLO26s at 640 and 1280 resolution. Removing DFL improves AP at both resolutions, with the gain increasing from +0.3 AP / +1.0 AP_L at 640 to +1.3 AP / +2.2 AP_L at 1280.

and faster head at the same accuracy (-0.3M parameters, -1.4 GFLOPs, -0.2 ms latency). To verify the effect more directly, we conduct controlled 640 and 1280 ablations using YOLO26s under a matched COCO training protocol, training the models with and without DFL independently from scratch under identical settings except for image size.

Table 3 confirms that, in the YOLO26s configuration, the DFL-free head improves AP at both 640 and 1280 resolution, and that the benefit becomes more pronounced at the higher resolution. Even at 640, removing DFL improves AP and AP_L, indicating that the finite support of DFL with `reg_max`=16 already restricts regression quality for the largest targets. The effect becomes stronger at 1280, where larger-object regression spans longer distances and therefore exposes this range limitation more clearly. In particular, the AP_L gain grows from +1.0 at 640 to +2.2 at 1280, supporting the claim that direct regression becomes more favorable as resolution increases. Together, the two tables indicate that DFL's localization benefit is replaceable by simpler components, while its finite-range cost becomes more limiting at higher resolution. Figure 4 further shows that the DFL-free head better preserves the full extent of large objects at 1280 resolution.

Optimizer	Epochs ↓	COCO mAP ↑
SGD	600	47.0
MuSGD	500	**47.4**

Table 4. MuSGD improves convergence speed and final accuracy when training YOLO26 from scratch on COCO. MuSGD reaches 47.4 mAP in 500 epochs versus 47.0 mAP for SGD in 600 epochs (16.7% fewer epochs).

4.3.2. MuSGD

We compare MuSGD against standard SGD when training YOLO26 from scratch on COCO. Table 4 shows that MuSGD reaches a higher final accuracy with a shorter schedule, improving mAP by +0.4 while reducing training from 600 to 500 epochs. This result supports MuSGD as a practical optimization improvement for the main detection setting.

To further validate MuSGD beyond detection, we also perform a controlled ImageNet classification comparison in which the backbone architecture and training recipe are held fixed and only the optimizer differs. The detailed results are provided in Sec. S1 of the supplementary materials and Table S1; the same trend holds there, indicating that the MuSGD advantage transfers beyond the detection setting.

4.3.3. Progressive Loss

Table 5 studies how the weighting between the one-to-many and one-to-one branches should evolve during training. The fixed equal-weight baseline reaches 46.4 E2E AP, while the best scheduled variant, $(0.8, 0.2) \rightarrow (0.1, 0.9)$, improves this to 46.7. Starting from $(1.0, 0.0)$ underperforms consistently, indicating that fully suppressing the one-to-one branch early in training is suboptimal. A near one-to-many-dominated start, such as $(0.9, 0.1)$, also hurts performance. Overall, the best schedule is the one that gives the one-



Figure 4. Qualitative comparison for large-object localization at 1280 resolution. From left to right, each row shows predictions from the model with DFL, predictions from the counterpart without DFL, and the ground-truth annotations. The DFL-free head better preserves full-object extent on large targets, supporting the quantitative gains reported in Table 3.

Start (o2m, o2o)	End (o2m, o2o)	mAP (E2E) ↑
(0.5, 0.5)	(0.5, 0.5)	46.4
(1.0, 0.0)	(0.1, 0.9)	46.4
(1.0, 0.0)	(0.2, 0.8)	46.4
(1.0, 0.0)	(0.3, 0.7)	46.3
(0.8, 0.2)	(0.1, 0.9)	**46.7**
(0.9, 0.1)	(0.1, 0.9)	46.3

Table 5. Ablation of Progressive Loss schedules on COCO using YOLO11s as the baseline. The default schedule $(0.8, 0.2) \rightarrow (0.1, 0.9)$ gives the best end-to-end mAP.

to-one branch nonzero supervision from the beginning and then gradually emphasizes it later.

Method	s_{ref}	AP ↑	AP_S	AP_M	AP_L
TAL (baseline)	–	46.6	29.0	51.4	**63.9**
STAL	8	46.6	27.7	51.6	63.8
STAL	16	**46.8**	**29.6**	**51.6**	63.8
STAL	32	46.5	28.3	51.3	63.7

Table 6. Ablation of the STAL reference size s_{ref} on COCO using YOLO11s as the baseline. Vanilla TAL achieves 46.6 AP. STAL with $s_{\text{ref}} = 16$ yields the best overall result at 46.8 AP (+0.2), with notable gains in AP_S (29.6 vs. 29.0). Both $s_{\text{ref}} = 8$ and $s_{\text{ref}} = 32$ match or slightly underperform the baseline, confirming that the next-stride reference size is the most effective choice.

4.3.4. STAL

Table 6 evaluates the reference size s_{ref} in STAL against vanilla TAL on COCO with YOLO11s as the baseline. Choosing the next stride level, $s_{\text{ref}} = 16$, gives the best overall result at 46.8 AP, improving the baseline by +0.2 AP and increasing AP_S from 29.0 to 29.6. Setting $s_{\text{ref}} = 8$ does not improve overall AP and noticeably reduces AP_S, which suggests that the adjustment is too weak to stabilize assignment for very small objects. Increasing the reference size further to 32 also degrades performance, indicating that excessive enlargement begins to distort the intended scale prior. Overall, these results support the default choice of $s_{\text{ref}} = 16$ in Section 4.1. Figure 5 further shows that STAL recovers small-object detections that the TAL baseline misses or localizes poorly.

4.4. Main Detection Results on COCO

Table 7 summarizes the released YOLO26 detection results on COCO [76]. We report both the standard validation mAP and the end-to-end mAP because YOLO26 supports two inference paths. The non-E2E numbers correspond to the one-to-many branch evaluated with NMS, while the E2E numbers correspond to the one-to-one branch used for true end-to-end inference without NMS.

End-to-End vs. NMS Flexibility. This distinction is practically important. The one-to-one head provides a simpler end-to-end deployment path and remains close to the NMS-based variant, trailing by only 0.6–0.8 AP across model scales. At the same time, YOLO26 does not force a single deployment mode: if the target platform or inference stack can execute NMS efficiently, the one-to-many head can still be preferred when the highest possible AP is the priority. Conversely, when deployment simplicity, tighter integration, or NMS-free inference is more valuable, the default one-to-one path provides a cleaner alternative.

Comparison with Recent Real-Time Detectors. A full grouped s/m/l/x comparison with recent real-time detectors

Model	Size (px)	mAP val 50–95	mAP (E2E) val 50–95	CPU ONNX (ms)	T4 TRT10 (ms)	Params (M)	FLOPs (B)
YOLO26n	640	40.9	40.1	38.9	1.7	2.4	5.4
YOLO26s	640	48.6	47.8	87.2	2.5	9.5	20.7
YOLO26m	640	53.1	52.5	220.0	4.7	20.4	68.2
YOLO26l	640	55.0	54.4	286.2	6.2	24.8	86.4
YOLO26x	640	57.5	56.9	525.8	11.8	55.7	193.9

Table 7. Released YOLO26 detection benchmarks on COCO from the Ultralytics repository [76]. 'mAP' denotes standard validation of the one-to-many branch with NMS ('end2end=False'), whereas 'mAP (E2E)' denotes true end-to-end validation of the default one-to-one branch. Speed values are the published CPU ONNX (Intel Xeon CPU @ 2.00 GHz) and T4 TensorRT10 benchmarks for the released models.

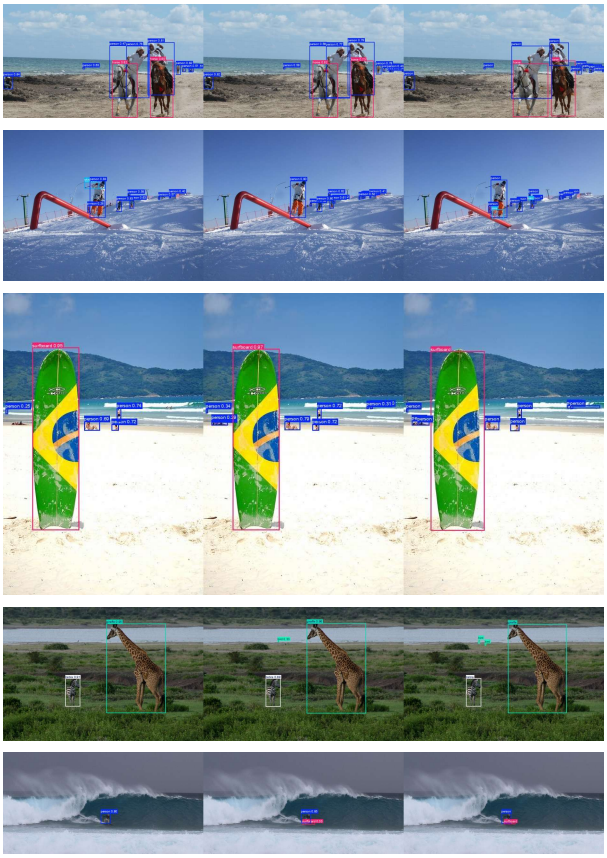


Figure 5. Qualitative comparison between TAL baseline (left), STAL with $s_{\text{ref}} = 16$ (middle), and ground-truth annotations (right) on COCO validation images. All predictions are generated at a confidence threshold of 0.25. STAL improves small-object detection by ensuring sufficient anchor coverage for tiny ground-truth boxes, leading to fewer missed detections and tighter localization that more closely matches the ground truth.

is provided in Sec. S4 of the supplementary materials and Table S8. At the standard NMS operating point, YOLO26 provides the strongest overall AP–latency trade-off in this comparison, achieving the best AP in the medium, large, and extra-large groups while remaining competitive in latency.

Method	mAP	AP^{50}
YOLO11s (baseline)	32.0	51.1
+ multi-scale proto module	32.4	51.7
+ auxiliary loss	32.7	52.0

Table 8. Effectiveness of module modification and auxiliary loss for instance segmentation on COCO.

4.5. Results on Various Vision Tasks

To show the robustness of our proposed methods, we present YOLO26 models on various vision tasks. Apart from task-specific optimization strategies, all other training and testing settings are kept consistent with those used for the detection models.

4.5.1. Instance Segmentation

As mentioned in Sec. 3.4.1, we adopt the multi-scale Proto Module and auxiliary semantic segmentation loss for instance segmentation. We evaluate the YOLO11s segmentation model on the COCO instance segmentation dataset to investigate the effect of each proposed method. As shown in Table 8, the multi-scale Proto Module improves the mAP from 32.0% to 32.4%, indicating that embedding higher-level semantic concepts into prototype maps can improve mask quality. We use equal BCE and Dice weights for the auxiliary loss, improving accuracy from 32.4% to 32.7% without compromising inference speed.

We integrate the proposed methods into YOLO26 to build the new segmentation models. Following the training policy of YOLO26 detection models, we adopt the Objects365-v1 pretrained weights and fine-tune on the COCO instance segmentation dataset. The full YOLO11 and YOLO26 family comparison is provided in Sec. S5 of the supplementary materials and Table S9. At the standard NMS operating point, YOLO26 improves box AP by **+1.6 to +2.5** and mask AP by **+2.4 to +3.7** over YOLO11 across

scales, while the end-to-end path remains close to the NMS-based variant.

4.5.2. Pose Estimation

w_{OKS}	w_{RLE}	mAP
48	0	61.5
48	1	60.8
24	**1**	**63.0**
24	2	62.5
12	1	62.6
0	1	61.9
0	2	62.4

Table 9. Ablation study on different weight configurations of the OKS and RLE loss, evaluated with YOLO26s pose model on the COCO keypoints validation set.

Pose estimation requires precise localization of keypoints under varying scales and appearance changes, and top-down pipelines remain sensitive to the quality of the underlying detector. We therefore use a combination of RLE loss and OKS loss while keeping the other training settings aligned with the detection models; see Sec. 3.4.2 for the loss formulation. Table 9 evaluates different loss weight settings on YOLO26s and shows that $w_{OKS} = 24$ and $w_{RLE} = 1$ gives the best mAP. The full YOLO11 and YOLO26 family comparison is provided in Sec. S5 of the supplementary materials and Table S10. In the E2E setting, YOLO26 improves pose AP by **+2.1 to +7.2** over YOLO11 across scales, while the end-to-end and NMS-based variants remain nearly equivalent, differing by at most 0.2 AP.

4.5.3. Oriented Bounding Box Detection

We evaluate our models on the DOTA-v1.0 [84] dataset, one of the largest and most commonly used datasets for oriented object detection, containing 2,806 images and 188,282 instances across 15 categories. We split the images into overlapping 1024×1024 crops and train our models on the training set.

We conduct ablation studies on the DOTA-v1.0 validation set to evaluate the effectiveness of the proposed methods, using YOLO26s without the angle loss as the baseline. As shown in Table 10, using the $[-45°, 135°)$ angle definition improves the mAP from 47.7 to 49.0, indicating that a more continuous angle parameterization is beneficial for OBB detection. Table 11 summarizes the ablations over the λ hyperparameter in the angle loss, where "—" indicates that the angle loss is not used. All configurations with the angle loss outperform the version without it, and $\lambda = 3$ gives the highest mAP at 50.2. The full YOLO11 and YOLO26 comparison on the DOTA-v1.0 test set is provided in Sec. S5 of the supplementary materials and Table S11. YOLO26 improves OBB mAP by **+2.5 to +3.4** over YOLO11 across scales, with larger AP_{75} gains of **+4.6 to +6.0**, indicating clearer improvements under stricter localization metrics.

Angle definition	mAP	AP^{50}
$(0, 90°]$	47.7	75.0
$[-45°, 135°)$	**49.0**	**75.4**

Table 10. Comparison of different angle definitions for OBB detection on DOTA-v1.0 validation set, using YOLO26s model without the angle loss as the backbone.

λ	mAP	AP^{50}
—	49.0	75.4
1	49.4	75.5
2	49.5	75.4
3	**50.2**	**76.0**
4	49.8	75.7
5	47.1	75.5

Table 11. Ablation study of different λ values used in the angle loss on the DOTA-v1.0 validation set, using YOLO26s as the backbone. "—" indicates no angle loss is used.

Table 12. Ablation study of YOLOE-26s-TP on the LVIS `minival` split. Starting from the YOLOE-11s-TP baseline, we progressively introduce the YOLOE-26 modifications described in Sec. 3.6 and report AP ($mAP_{50:95}$) under both end-to-end (E2E) and non-end-to-end (non-E2E) evaluation protocols.

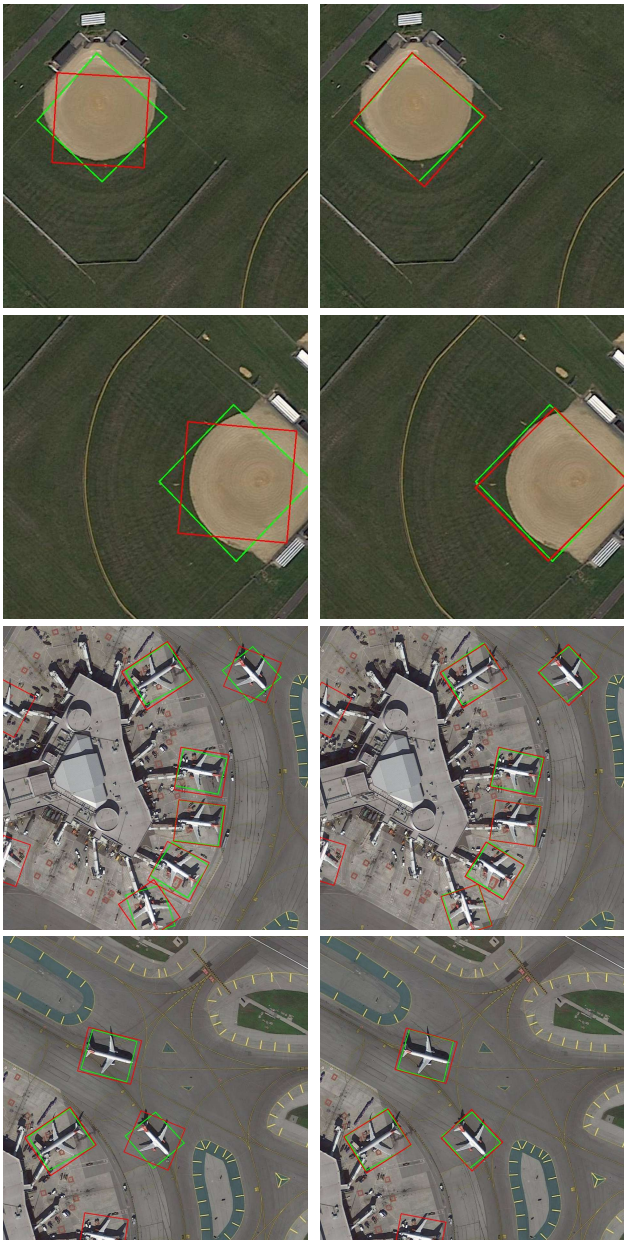
Backbone	Decoupled Seg.	Text Enc. Upgrade	Data Engine	AP_{E2E}	$AP_{non\text{-}E2E}$
v8s	—	—	—	—	27.9
11s	—	—	—	—	27.5
26s	—	—	—	27.8	29.0
26s	✓	—	—	28.5	29.5
26s	✓	✓	—	28.8	29.7
26s	✓	✓	✓	**29.9**	**31.0**

Additional qualitative examples are provided in Fig. 6. They show that YOLO26 produces visibly better angle predictions than YOLO11 on square rotated objects, consistent with the larger gains observed in AP75 and overall mAP than in AP50.

4.6. YOLOE-26 Results

4.6.1. Ablation Study

Table 12 isolates the four YOLOE-26 modifications introduced in Sec. 3.6: Backbone upgrade, Decoupled segmentation training, Text encoder upgrade, and Data engine. Unless otherwise stated, all rows use the same TP training setup with Objects365-v1 [63] and GoldG grounding data [96], including GQA [23] and Flickr30k [56] with COCO images excluded, and use MobileCLIP as the default text encoder. We report AP under both end-to-end (E2E)



YOLO11x-obb YOLO26x-obb

Figure 6. Qualitative comparisons between YOLO26x-obb and YOLO11x-obb models for square rotated objects detection. Green boxes are ground truth, while red boxes indicate predictions. YOLO26 demonstrates better angle predictions over YOLO11 on square objects.

and non-end-to-end (non-E2E) evaluation protocols.

For reference, the YOLOE-v8s-TP and YOLOE-11s-TP baselines achieve 27.9 and 27.5 $AP_{\text{non-E2E}}$ respectively. We begin from the latter. Applying the Backbone upgrade by replacing with pretrained YOLO26s weights yields 27.8/29.0 AP (E2E/non-E2E), a notable +1.5 AP gain under the non-E2E protocol. Enabling Decoupled segmentation training further improves E2E AP by +0.7 to 28.5 and non-E2E AP by +0.5 to 29.5, supporting the view that removing the auxiliary segmentation objective reduces task interference. The Text encoder upgrade from MobileCLIP to MobileCLIP2 brings an additional +0.3/+0.2 AP (E2E/non-E2E), reaching 28.8/29.7. Finally, the Data engine described in Sec. 3.6 yields the largest single improvement of +1.1/+1.3 AP, reaching the final 29.9/31.0 AP result. This gain is likely due to the improved grounding data quality and diversity, which better supports the open-vocabulary detection capability of the model.

4.6.2. Prompt-based evaluation

The full text-prompted and visual-prompted detection comparison is provided in Sec. S7 of the supplementary materials and Table S12. Overall, YOLOE-26 improves over earlier YOLOE variants across scales under both text and visual prompting. Under text prompting, the Non-E2E branch improves AP by **+1.9–2.9** over YOLOE-v8 and **+2.4–3.3** over YOLOE-11 across the s/m/l models; under visual prompting, the corresponding gains are **+2.1–2.9** and **+2.3–2.6**. The largest model achieves the strongest prompt-based results, reaching **40.6 AP** with text prompts and **38.5 AP** with visual prompts, while the lightweight YOLOE-26n still reaches 24.7 AP with only 3.9M parameters. For text-prompted inference, the end-to-end head remains close to the Non-E2E variant, trailing by at most 1.1 AP across scales. Under visual prompting, the gap ranges from 1.0 to 2.6 AP.

For zero-shot segmentation, the full comparison is provided in Sec. S7 of the supplementary materials and Table S13. YOLOE-26 likewise improves zero-shot mask prediction across the s/m/l models under both text and visual prompts, with AP^m gains of **+1.3–3.0** over YOLOE-v8 and **+1.8–2.9** over YOLOE-11. YOLOE-26x reaches the best overall result of **27.4 / 26.7 AP^m** under text/visual prompting, indicating that the YOLO26 backbone and training refinements benefit zero-shot segmentation as well as open-vocabulary detection.

4.6.3. Prompt-free evaluation

The full prompt-free comparison is provided in Sec. S7 of the supplementary materials and Table S14. YOLOE-26 remains competitive across the model family in the prompt-free setting. In the standard Non-E2E setting, YOLOE-26 improves AP by **+0.8–1.7** over YOLOE-v8 and **+0.9–2.0** over YOLOE-11 across the s/m/l models, reaching up to **31.1 AP** on LVIS `minival`. The E2E head stays within **0.7–1.2 AP** of the corresponding Non-E2E models while removing post-processing.

5. Conclusion

We presented YOLO26, a unified real-time vision model family that combines a dual-head NMS-free architecture with MuSGD, Progressive Loss, and STAL to improve the accuracy–latency trade-off across five model scales. By removing DFL and strengthening optimization and label assignment, YOLO26 achieves a lighter detection head while preserving localization quality. Beyond detection, the YOLO26 family combines task-specific refinements (multi-scale prototype fusion for segmentation, uncertainty-aware keypoint regression for pose, and revised OBB parameterization with dedicated angle supervision) with the shared detector improvements, improving over YOLO11 by up to +2.5 box AP and +3.7 mask AP on COCO instance segmentation, +7.2 AP on COCO pose estimation, and +3.4 mAP on DOTA-v1.0 OBB detection, while maintaining a unified training and deployment pipeline with native export support across the 19 non-PyTorch targets exposed by Ultralytics. On COCO, YOLO26 reaches 40.9–57.5 mAP at 1.7–11.8 ms T4 TensorRT latency. YOLOE-26 further extends the family to open-vocabulary detection, where YOLOE-26x reaches 40.6 AP on LVIS minival under text prompting and 38.5 AP under visual prompting, while YOLOE-26 also remains competitive in the prompt-free setting (up to 31.1 AP), showing that the stronger YOLO26 detector and the additional open-vocabulary refinements jointly improve performance in the open-vocabulary setting. Future work includes broader evaluation beyond COCO-centric benchmarks and further exploration of learned or task-adaptive loss-schedule shapes beyond the linear $\alpha(t)$ used here, as well as pretraining beyond Objects365-v1, including web-scale or grounding-style corpora.

References

[1] Apple. Core ML Tools. https://github.com/apple/coremltools, 2024. Accessed: 2026-02-05. 2

[2] Daniel Bolya, Chong Zhou, Fanyi Xiao, and Yong Jae Lee. Yolact: Real-time instance segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9157–9166, 2019. 4, 8

[3] Gary Bradski. The opencv library. *Dr. Dobb's Journal of Software Tools*, 2000. 9

[4] Zhe Cao, Gines Hidalgo, Tomas Simon, Shih-En Wei, and Yaser Sheikh. OpenPose: Realtime multi-person 2D pose estimation using part affinity fields. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(1):172–186, 2021. 4

[5] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *Computer Vision – ECCV 2020*, pages 213–229. Springer, Cham, 2020. 2, 3

[6] Qiang Chen, Xiaokang Chen, Jian Wang, Shan Zhang, Kun Yao, Haocheng Feng, Junyu Han, Errui Ding, Gang Zeng, and Jingdong Wang. Group DETR: Fast DETR training with group-wise one-to-many assignment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6633–6642, 2023. 4

[7] Qiang Chen, Xiangbo Su, Xinyu Zhang, Jian Wang, Jiahui Chen, Yunpeng Shen, Chuchu Han, Ziliang Chen, Weixiang Xu, Fanrong Li, Shan Zhang, Kun Yao, Errui Ding, Gang Zhang, and Jingdong Wang. LW-DETR: A transformer replacement to YOLO for real-time detection. *arXiv preprint arXiv:2406.03459*, 2024. 4

[8] Bowen Cheng, Ishan Misra, Alexander G. Schwing, Alexander Kirillov, and Rohit Girdhar. Masked-attention mask transformer for universal image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1290–1299, 2022. 4

[9] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 5, 29

[10] Jian Ding, Nan Xue, Yang Long, Gui-Song Xia, and Qikai Lu. Learning RoI transformer for oriented object detection in aerial images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2849–2858, 2019. 4

[11] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real-valued non-volume preserving (Real NVP) transformations. In *International Conference on Learning Representations (ICLR)*, 2017. 8

[12] Kaiwen Duan, Song Bai, Lingxi Xie, Honggang Qi, Qingming Huang, and Qi Tian. Centernet: Keypoint triplets for object detection. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 6569–6578, 2019. 3

[13] Fartash Faghri, Pavan Kumar Anasosalu Vasu, Cem Koc, Vaishaal Shankar, Alexander T. Toshev, Oncel Tuzel, and Hadi Pouransari. MobileCLIP2: Improving multi-modal reinforced training. *Transactions on Machine Learning Research (TMLR)*, 2025. 3, 11

[14] Chengjian Feng, Yujie Zhong, Yu Gao, Matthew R. Scott, and Weilin Huang. Tood: Task-aligned one-stage object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3490–3499, 2021. 2, 6

[15] Ross Girshick. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 1440–1448, 2015. 3

[16] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 580–587, 2014. 3

[17] Google. TensorFlow Lite. https://ai.google.dev/edge/litert, 2024. Accessed: 2026-02-05. 2

[18] Jiaming Han, Jian Ding, Jie Li, and Gui-Song Xia. Align deep features for oriented object detection. *IEEE Transactions on Geoscience and Remote Sensing*, 60:1–11, 2022. 5

[19] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017. 4

[20] Shihua Huang, Yongjie Hou, Longfei Liu, Xuanlong Yu, and Xi Shen. Real-time object detection meets DINOv3. *arXiv preprint arXiv:2509.20787*, 2025. 4

[21] Shihua Huang, Zhichao Lu, Xiaodong Cun, Yongjun Yu, Xiao Zhou, and Xi Shen. DEIM: DETR with improved matching for fast convergence. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15162–15171, 2025. 2, 4, 27

[22] Xinyu Huang, Yi-Jie Huang, Youcai Zhang, Weiwei Tian, Rui Feng, Yuejie Zhang, Yanchun Xie, Yaqian Li, and Lei Zhang. Open-set image tagging with multi-grained text supervision. In *Proceedings of the 33rd ACM International Conference on Multimedia*, pages 4117–4126, 2025. 11

[23] Drew A. Hudson and Christopher D. Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6700–6709, 2019. 15, 24, 30

[24] Intel. Intel Distribution of OpenVINO Toolkit. https://github.com/openvinotoolkit/openvino, 2024. Accessed: 2026-02-05. 2

[25] Ding Jia, Yuhui Yuan, Haodi He, Xiaopei Wu, Haojun Yu, Weihong Lin, Lei Sun, Chao Zhang, and Han Hu. DETRs with hybrid matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19702–19712, 2023. 4

[26] Qing Jiang, Feng Li, Zhaoyang Zeng, Tianhe Ren, Shilong Liu, and Lei Zhang. T-Rex2: Towards generic object detection via text-visual prompt synergy. *arXiv preprint arXiv:2403.14610*, 2024. 5, 29

[27] Tao Jiang, Peng Lu, Li Zhang, Ningsheng Ma, Rui Han, Chengqi Lyu, Yining Li, and Kai Chen. RTMPose: Real-time multi-person pose estimation based on MMPose. *arXiv preprint arXiv:2303.07399*, 2023. 4

[28] Glenn Jocher et al. ultralytics/yolov5: Initial release, 2020. 2, 3

[29] Feng Li, Hao Zhang, Shilong Liu, Jian Guo, Lionel M. Ni, and Lei Zhang. DN-DETR: Accelerate DETR training by introducing query denoising. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13619–13627, 2022. 4

[30] Feng Li, Hao Zhang, Peize Sun, Xueyan Zou, Shilong Liu, Jianwei Yang, Chunyuan Li, Lei Zhang, and Jianfeng Gao. Semantic-sam: Segment and recognize anything at any granularity. *arXiv preprint arXiv:2307.04767*, 2023. 5

[31] Feng Li, Hao Zhang, Huaizhe Xu, Shilong Liu, Lei Zhang, Lionel M. Ni, and Heung-Yeung Shum. Mask dino: Towards a unified transformer-based framework for object detection and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3041–3050, 2023. 4

[32] Feng Li, Qing Jiang, Hao Zhang, Tianhe Ren, Shilong Liu, Xueyan Zou, Huaizhe Xu, Hongyang Li, Jianwei Yang, Chunyuan Li, Lei Zhang, and Jianfeng Gao. Visual in-context prompting. In *CVPR*, pages 12861–12871, 2024. 5

[33] Jiefeng Li, Siyuan Bian, Ailing Zeng, Can Wang, Bo Pang, Wentao Liu, and Cewu Lu. Human pose regression with residual log-likelihood estimation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11025–11034, 2021. 3, 4, 8

[34] Liunian Harold Li, Pengchuan Zhang, Haotian Zhang, Jianwei Yang, Chunyuan Li, Yiwu Zhong, Lijuan Wang, Lu Yuan, Lei Zhang, Jenq-Neng Hwang, Kai-Wei Chang, and Jianfeng Gao. Grounded language-image pre-training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. 5, 29

[35] Xiang Li, Wenhai Wang, Lijun Wu, Shuo Chen, Xiaolin Hu, Jun Li, Jinhui Tang, and Jian Yang. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *Advances in Neural Information Processing Systems*, pages 21002–21012. Curran Associates, Inc., 2020. 6

[36] Yuxuan Li, Qibin Hou, Zhaohui Zheng, Ming-Ming Cheng, Jian Yang, and Xiang Li. LSKNet: A foundation lightweight backbone for remote sensing. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4829–4840, 2023. 5

[37] Zijun Liao, Yian Zhao, Xin Shan, Yu Yan, Chang Liu, Lei Lu, Xiangyang Ji, and Jie Chen. RT-DETRv4: Painlessly furthering real-time object detection with vision foundation models. *arXiv preprint arXiv:2510.25257*, 2025. 4

[38] Chuang Lin, Yi Jiang, Lizhen Qu, Zehuan Yuan, and Jianfei Cai. Generative region-language pretraining for open-ended object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 5, 26, 30

[39] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017. 2, 3

[40] Jingyuan Liu, Jianlin Su, Xingcheng Yao, Zhejun Jiang, Guokun Lai, Yulun Du, Yidao Qin, Weixin Xu, Enzhe Lu, Junjie Yan, Yanru Chen, Huabin Zheng, Yibo Liu, Bohong Yin, Weiran He, Han Zhu, Yuzhi Wang, Jianzhou Wang, Mengnan Dong, Zheng Zhang, Yongsheng Kang, Hao Zhang, Xinran Xu, Yutao Zhang, Yuxin Wu, Xinyu Zhou, and Zhilin Yang. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025. 2, 7

[41] Lihao Liu, Juexiao Zeng, Xu Gao, Baize Yan, Yi Luo, Guang Wang, and Yunzhe Zhuge. Yolo-uniow: Efficient universal open-world object detection. *arXiv preprint arXiv:2412.20645*, 2024. 5

[42] Shilong Liu, Feng Li, Hao Zhang, Xiao Yang, Xianbiao Qi, Hang Su, Jun Zhu, and Lei Zhang. DAB-DETR: Dynamic anchor boxes are better queries for DETR. In *International Conference on Learning Representations (ICLR)*, 2022. 4

[43] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024. 5, 29

[44] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. Ssd: Single shot multibox detector. In *European conference on computer vision*, pages 21–37. Springer, 2016. 2, 3

[45] Jeffri M. Llerena, Luis Felipe Zeni, Lucas N. Kristen, and Claudio Jung. Gaussian bounding boxes and probabilistic intersection-over-union for object detection. *arXiv preprint arXiv:2106.06072*, 2021. 5, 9

[46] Wenyu Lv, Yian Zhao, Qinyao Chang, Kui Huang, Guanzhong Wang, and Yi Liu. Rt-detrv2: Improved baseline with bag-of-freebies for real-time detection transformer. *arXiv preprint arXiv:2407.17140*, 2024. 27

[47] Jianqi Ma, Weiyuan Shao, Hao Ye, Li Wang, Hong Wang, Yingbin Zheng, and Xiangyang Xue. Arbitrary-oriented scene text detection via rotation proposals. *IEEE Transactions on Multimedia*, 20(11):3111–3122, 2018. 4

[48] Debapriya Maji, Soyeb Nagori, Manu Mathew, and Deepak Poddar. YOLO-Pose: Enhancing YOLO for multi person pose estimation using object keypoint similarity loss. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 2637–2646, 2022. 4, 8

[49] Matthias Minderer, Alexey Gritsenko, Austin Stone, Maxim Neumann, Dirk Weissenborn, Alexey Dosovitskiy, Aravindh Mahendran, Anurag Arnab, Mostafa Dehghani, Zhuoran Shen, Xiao Wang, Xiaohua Zhai, Thomas Kipf, and Neil Houlsby. Simple open-vocabulary object detection with vision transformers. In *ECCV*, 2022. 5

[50] Moonshot AI. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025. 7

[51] ncnn Contributors. ncnn documentation. https://ncnn.readthedocs.io/en/latest/, 2026. Official documentation for the ncnn inference framework. Accessed: 2026-04-10. 2

[52] Alejandro Newell, Kaiyu Yang, and Jia Deng. Stacked hourglass networks for human pose estimation. In *Computer Vision – ECCV 2016*, pages 483–499. Springer, Cham, 2016. 4

[53] NVIDIA. NVIDIA TensorRT: High-performance deep learning inference sdk. https://github.com/NVIDIA/TensorRT, 2024. Accessed: 2026-02-05. 2

[54] ONNX Contributors. ONNX: Open neural network exchange. https://github.com/onnx/onnx, 2019. Accessed: 2026-02-05. 2

[55] Yansong Peng, Hebei Li, Peixi Wu, Yueyi Zhang, Xiaoyan Sun, and Feng Wu. D-FINE: Redefine regression task of DETRs as fine-grained distribution refinement. In *International Conference on Learning Representations (ICLR)*, 2025. 2, 4, 6, 27

[56] Bryan A. Plummer, Liwei Wang, Chris M. Cervantes, Juan C. Caicedo, Julia Hockenmaier, and Svetlana Lazebnik. Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2641–2649, 2015. 15, 24, 30

[57] PyTorch Contributors. Executorch documentation. https://docs.pytorch.org/executorch/stable/, 2026. Official documentation for ExecuTorch. Accessed: 2026-04-10. 2

[58] PyTorch Contributors. torch.optim.sgd. https://docs.pytorch.org/docs/stable/generated/torch.optim.SGD.html, 2026. Accessed: 2026-02-02. 7

[59] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018. 2, 3

[60] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster r-cnn: Towards real-time object detection with region proposal networks. *Advances in neural information processing systems*, 28, 2015. 2, 3

[61] Tianhe Ren, Shilong Liu, Zhaoyang Zeng, Hao Lin, Feng Li, Hao Tang, Qing Jiang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO 1.5: Advance the "edge" of open-set object detection. *arXiv preprint arXiv:2405.10300*, 2024. 29

[62] Isaac Robinson, Peter Robicheaux, Matvei Popov, Deva Ramanan, and Neehar Peri. RF-DETR: Neural architecture search for real-time detection transformers. *arXiv preprint arXiv:2511.09554*, 2025. 2, 4

[63] Shuai Shao, Zeming Li, Tianyuan Zhang, Chao Peng, Gang Yu, Jing Li, Xiangyu Zhang, and Jian Sun. Objects365: A large-scale, high-quality dataset for object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8425–8434, 2019. 11, 15, 24, 30

[64] Ke Sun, Bin Xiao, Dong Liu, and Jingdong Wang. Deep high-resolution representation learning for human pose estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5693–5703, 2019. 4

[65] Yunjie Tian, Qixiang Ye, and David Doermann. Yolov12: Attention-centric real-time object detectors. In *Advances in Neural Information Processing Systems*, 2025. 27

[66] Zhi Tian, Chunhua Shen, Hao Chen, and Tong He. Fcos: Fully convolutional one-stage object detection. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9627–9636, 2019. 2, 3

[67] Zhi Tian, Chunhua Shen, and Hao Chen. Conditional convolutions for instance segmentation. In *Computer Vision – ECCV 2020*. Springer, Cham, 2020. 4

[68] Alexander Toshev and Christian Szegedy. DeepPose: Human pose estimation via deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1653–1660, 2014. 4

[69] Ultralytics. Explore ultralytics yolov8. https://docs.ultralytics.com/models/yolov8/, 2023. Online documentation (no formal paper). 2, 3, 4, 5, 6

[70] Ultralytics. Ultralytics yolo11. https://docs.ultralytics.com/models/yolo11/, 2024. Online documentation (no formal paper). Accessed: 2026-02-02. 2, 4, 5, 6, 8, 27

[71] Ultralytics. Ultralytics export mode. https://docs.ultralytics.com/modes/export/, 2026. Online documentation describing supported export formats and deployment targets. Accessed: 2026-04-10. 2, 9

[72] Ultralytics. Ultralytics predict mode. https://docs.ultralytics.com/modes/predict/, 2026. Online documentation describing model inference workflows. Accessed: 2026-06-01. 9

[73] Ultralytics. Ultralytics computer vision tasks. https://docs.ultralytics.com/tasks/, 2026. Online documentation describing supported computer vision tasks. Accessed: 2026-06-01. 2, 9

[74] Ultralytics. Ultralytics train mode. https://docs.ultralytics.com/modes/train/, 2026. Online documentation describing model training workflows. Accessed: 2026-06-01. 9

[75] Ultralytics. Ultralytics val mode. https://docs.ultralytics.com/modes/val/, 2026. Online documentation describing model validation workflows. Accessed: 2026-06-01. 9

[76] Ultralytics. Ultralytics yolo26. https://docs.ultralytics.com/models/yolo26/, 2026. Online documentation and released benchmark tables. Accessed: 2026-03-13. 13, 14

[77] Pavan Kumar Anasosalu Vasu, Hadi Pouransari, Fartash Faghri, Raviteja Vemulapalli, and Oncel Tuzel. MobileCLIP: Fast image-text models through multi-modal reinforced training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 10, 11

[78] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, et al. Yolov10: Real-time end-to-end object detection. *Advances in Neural Information Processing Systems*, 37:107984–108011, 2024. 2, 3, 6, 27

[79] Ao Wang, Lihao Liu, Hui Chen, Zijia Lin, Jungong Han, and Guiguang Ding. YOLOE: Real-time seeing anything.

In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025. 3, 5, 10, 24, 29, 30

[80] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7464–7475, 2023. 4

[81] Chien-Yao Wang, I-Hau Yeh, and Hong-Yuan Mark Liao. Yolov9: Learning what you want to learn using programmable gradient information. In *European conference on computer vision*, pages 1–21. Springer, 2024. 6, 27

[82] Xinlong Wang, Rufeng Zhang, Tao Kong, Lei Li, and Chunhua Shen. Solov2: Dynamic and fast instance segmentation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. 4

[83] Jialian Wu, Jianfeng Wang, Zhengyuan Yang, Zhe Gan, Zicheng Liu, Junsong Yuan, and Lijuan Wang. Grit: A generative region-to-text transformer for object understanding. *arXiv preprint arXiv:2212.00280*, 2022. 5

[84] Gui-Song Xia, Xiang Bai, Jian Ding, Zhen Zhu, Serge Belongie, Jiebo Luo, Mihai Datcu, Marcello Pelillo, and Liangpei Zhang. Dota: A large-scale dataset for object detection in aerial images. In *CVPR*, pages 3974–3983, 2018. 15

[85] Bin Xiao, Haiping Wu, and Yichen Wei. Simple baselines for human pose estimation and tracking. In *Computer Vision – ECCV 2018*, pages 472–487. Springer, Cham, 2018. 4

[86] Xingxing Xie, Gong Cheng, Jiabao Wang, Xiwen Yao, and Junwei Han. Oriented R-CNN for object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3520–3529, 2021. 4

[87] Yufei Xu, Jing Zhang, Qiming Zhang, and Dacheng Tao. ViTPose: Simple vision transformer baselines for human pose estimation. In *Advances in Neural Information Processing Systems*, pages 38571–38584, 2022. 4

[88] Xue Yang and Junchi Yan. Arbitrary-oriented object detection with circular smooth label. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 677–694, 2020. 4

[89] Xue Yang, Junchi Yan, Ming Qi, Wentao Wang, Xiaopeng Zhang, and Qi Tian. Rethinking rotated object detection with Gaussian wasserstein distance loss. In *Proceedings of the International Conference on Machine Learning (ICML)*, pages 11580–11591, 2021. 4

[90] Xue Yang, Gefan Liu, Jirui Yang, Jie Yi, Wentong Liao, Tao He, Jian Zhang, and Junchi Yan. The KFIoU loss for rotated object detection. In *International Conference on Learning Representations (ICLR)*, 2023. 4

[91] Lewei Yao, Jianhua Han, Youpeng Wen, Xiaodan Liang, Dan Xu, Wei Zhang, Zhenguo Li, Chunjing Xu, and Hang Xu. DetCLIP: Dictionary-enriched visual-concept paralleled pre-training for open-world detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 3, 29

[92] Lewei Yao, Jianhua Han, Xiaodan Liang, Dan Xu, Wei Zhang, Zhenguo Li, and Hang Xu. Detclipv3: Towards versatile generative open-vocabulary object detection. *arXiv preprint arXiv:2404.09216*, 2024. 5

[93] Xinyi Yu, Jiangping Lu, Mi Lin, Libo Zhou, and Linlin Ou. Mkiou loss: Toward accurate oriented object detection in aerial images. *Journal of Electronic Imaging*, 32(3): 033030–033030, 2023. 5

[94] Yi Yu and Feipeng Da. Phase-shifting coder: Predicting accurate orientation in oriented object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 13354–13363, 2023. 5

[95] Yuhang Zang, Wei Li, Kaiyang Zhou, Chen Huang, and Chen Change Loy. Open-vocabulary detr with conditional matching. In *ECCV*, pages 106–122, 2022. 5

[96] Haotian Zhang, Pengchuan Zhang, Xiaowei Hu, Yen-Chun Chen, Liunian Harold Li, Xiyang Dai, Lijuan Wang, Lu Yuan, Jenq-Neng Hwang, and Jianfeng Gao. GLIPv2: Unifying localization and vision-language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. 15, 29

[97] Hao Zhang, Feng Li, Shilong Liu, Lei Zhang, Hang Su, Jun Zhu, Lionel Ni, and Heung-Yeung Shum. DINO: DETR with improved denoising anchor boxes for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2023. 4

[98] Yian Zhao, Wenyu Lv, Shangliang Xu, Jinman Wei, Guanzhong Wang, Qingqing Dang, Yi Liu, and Jie Chen. DETRs beat YOLOs on real-time object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16965–16974, 2024. 2, 4

[99] Yue Zhou, Xue Yang, Gefan Zhang, Jiabao Wang, Yanyi Liu, Liping Hou, Xue Jiang, Xingzhao Liu, Junchi Yan, Chengqi Lyu, Wenwei Zhang, and Kai Chen. Mmrotate: A rotated object detection benchmark using pytorch. In *Proceedings of the 30th ACM International Conference on Multimedia*, 2022. 9

[100] Xizhou Zhu, Weijie Su, Lewei Lu, Bin Li, Xiaogang Wang, and Jifeng Dai. Deformable DETR: Deformable transformers for end-to-end object detection. In *International Conference on Learning Representations (ICLR)*, 2021. 3

[101] Xueyan Zou, Jianwei Yang, Hao Zhang, Feng Li, Linjie Li, Jianfeng Wang, Lijuan Wang, Jianfeng Gao, and Yong Jae Lee. Segment everything everywhere all at once. *NeurIPS*, 2024. 5

Model	Optimizer	Top-1 ↑	Params (M)	FLOPs (B)
YOLO11n-cls	SGD	70.0	1.6	3.3
YOLO26n-cls	MuSGD	**71.4**	1.6	3.3
YOLO11s-cls	SGD	75.4	5.5	12.1
YOLO26s-cls	MuSGD	**76.0**	5.5	12.1
YOLO11m-cls	SGD	77.3	10.4	39.3
YOLO26m-cls	MuSGD	**78.1**	10.4	39.3
YOLO11l-cls	SGD	78.3	12.9	49.4
YOLO26l-cls	MuSGD	**79.0**	12.9	49.4
YOLO11x-cls	SGD	79.5	28.4	110.4
YOLO26x-cls	MuSGD	**79.9**	28.4	110.4

Table S1. Controlled ImageNet classification comparison for MuSGD. Within each model pair, the backbone architecture and training recipe are matched, so the reported differences isolate the effect of the optimizer. MuSGD consistently improves top-1 accuracy across all scales.

S. Supplementary Materials

S1. MuSGD Transfer to Classification

To test whether the MuSGD advantage transfers beyond detection, we also perform a controlled ImageNet classification comparison, summarized in Table S1. For each scale, the paired models share the same backbone architecture and training recipe; only the optimizer differs.

S2. Architecture Visualizations

For completeness, we provide two supplementary architecture figures that complement the main methodology section. Figure S1 presents the overall YOLO26 model architecture, while Figure S2 details the building blocks used to assemble the network. Together, these visualizations make the macro-architecture and the underlying module composition explicit.

S3. Training Recipes

Note on loss configuration fields. In the released Ultralytics training configuration, the `dfl` field controls the bounding-box regression loss gain. For YOLO11 and prior versions this governs the Distribution Focal Loss. In YOLO26, DFL is removed and direct regression with an L1 loss is used instead; the `dfl` field is retained for backward compatibility and now scales the L1 regression loss gain. Thus, a nonzero `dfl` value in the tables below does not indicate that DFL is active.

Objects365-v1 Pretraining. We first summarize the Objects365-v1 pretraining stage. Unless otherwise stated, Tables S2–S4 are transcribed directly from the `train_args` stored in the released Objects365-v1 pretraining checkpoints, with most floating-point values rounded to two decimals for readability. We retain the original precision for `lr0` and `weight_decay`. The pretraining stage uses 150 epochs at 640×640 resolution with batch size 128 and size-specific augmentation and internal settings.

COCO Fine-Tuning. We next summarize the COCO fine-tuning stage. Unless otherwise stated, Tables S5–S7 are transcribed directly from the `train_args` stored in the released COCO checkpoints, with most floating-point values rounded to two decimals for readability. We retain the original precision for `lr0` and `weight_decay`, and use ~ 0 when a positive value rounds to zero at two-decimal precision. The released detection checkpoints use end-to-end training at 640×640 resolution with batch size 128, are initialized from the Objects365-v1 pretraining checkpoints above, and use size-specific hyperparameters obtained by evolutionary search.

S4. Comparison with Recent Real-Time Detectors

Table S8 provides the full grouped s/m/l/x comparison with recent real-time detectors. It complements Sec. 4.4 by giving the complete per-scale breakdown used to position YOLO26 in the main paper.

S5. Additional Task Benchmarks

This subsection collects the full YOLO11-versus-YOLO26 benchmark tables for the additional task results discussed in Sec. 4.5. We place these full model-family comparisons in the supplementary materials so that the main paper can focus on the task-specific ablations and methodological takeaways.

Instance Segmentation. Table S9 reports the full COCO instance segmentation benchmark comparison between YOLO11 and YOLO26.

Pose Estimation. Table S10 reports the full COCO pose estimation benchmark comparison between YOLO11 and YOLO26.

Oriented Bounding Box Detection. Table S11 reports the full DOTA-v1.0 OBB benchmark comparison between YOLO11 and YOLO26.

S6. YOLOE-26 Implementation Details

All YOLOE-26 models are trained with a total batch size of 256 across multiple GPUs. The training follows a four-stage pipeline: text-prompt training (TP) is conducted first, and its best checkpoint then serves as the initialization for three parallel downstream branches—visual prompt (VP), prompt-free (PF), and segmentation (SEG).

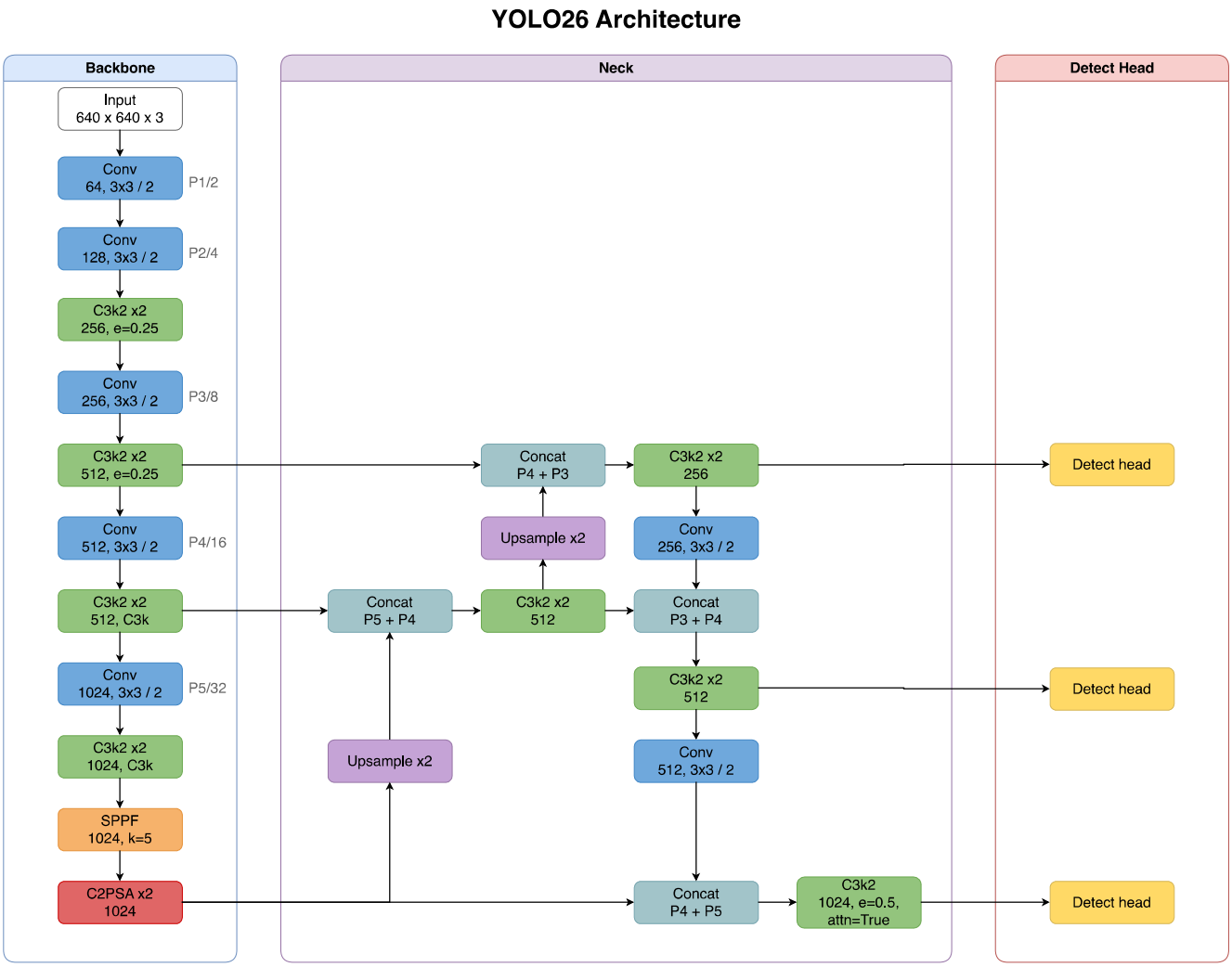


Figure S1. Ultralytics YOLO26 architecture diagram. The figure summarizes the end-to-end model structure, including the shared backbone and neck, the task heads, and the high-level information flow across the network.

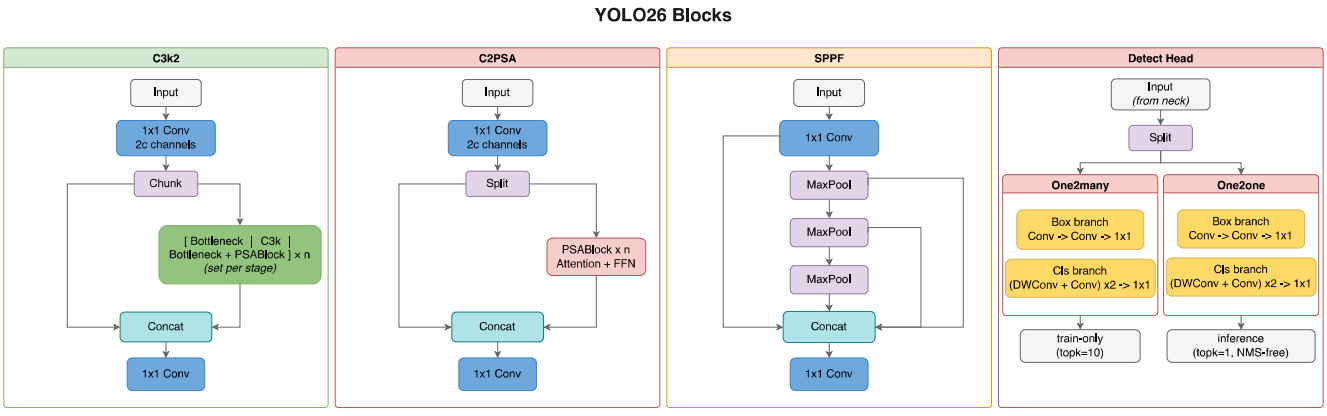


Figure S2. Building blocks used to assemble the Ultralytics YOLO26 architecture. The figure details the constituent modules that compose the full network shown in Fig. S1.

Setting	YOLO26n	YOLO26s	YOLO26m	YOLO26l	YOLO26x
Optimizer and Schedule					
optimizer	MuSGD	MuSGD	MuSGD	MuSGD	MuSGD
end2end	–	–	–	–	–
imgsz	640	640	640	640	640
batch	128	128	128	128	128
epochs	150	150	150	150	150
lr0	0.01	0.01	0.01	0.01	0.01
lrf	0.01	0.01	0.01	0.01	0.01
momentum	0.94	0.94	0.94	0.94	0.94
weight_decay	0.0005	0.0005	0.0005	0.0005	0.0005
warmup_epochs	1	1	1	1	2
close_mosaic	8	8	8	8	8
Loss Weights					
box	7.50	7.50	7.50	7.50	7.50
cls	0.50	0.50	0.50	0.50	0.50
dfl	6.00	6.00	6.00	6.00	6.00

Table S2. Official optimizer, schedule, and loss settings for the Objects365-v1 pretraining checkpoints.

Setting	YOLO26n	YOLO26s	YOLO26m	YOLO26l	YOLO26x
mosaic	1.00	1.00	1.00	1.00	1.00
mixup	0.00	0.05	0.15	0.15	0.20
copy_paste	0.10	0.15	0.40	0.50	0.60
scale	0.50	0.90	0.90	0.90	0.90
fliplr	0.50	0.50	0.50	0.50	0.50
degrees	0.00	0.00	0.00	0.00	0.00
shear	0.00	0.00	0.00	0.00	0.00
translate	0.10	0.10	0.10	0.10	0.10
hsv_h	0.02	0.02	0.02	0.02	0.02
hsv_s	0.70	0.70	0.70	0.70	0.70
hsv_v	0.40	0.40	0.40	0.40	0.40
bgr	0.00	0.00	0.00	0.00	0.00

Table S3. Official size-specific augmentation settings for the Objects365-v1 pretraining checkpoints.

TP stage. Each backbone is initialized from a YOLO26 checkpoint pretrained on Objects365-v1 for 150 epochs. We use the MuSGD optimizer with an initial learning rate of 1.25×10^{-3}, a final learning rate ratio $\mathrm{lr}_f = 0.5$, momentum 0.9, and weight decay 5×10^{-4} (7×10^{-4} for the S scale). Mosaic augmentation is disabled for the last 2 epochs (close_mosaic = 2). The number of training epochs is scale-dependent: 30 for n/s, 25 for m, 20 for l, and 15 for x. Data augmentation strength also scales with model capacity: copy-paste probability increases from 0.1 (N) to 0.6 (X), and mixup from 0.0 (N) to 0.2 (X). The L and X scales additionally adopt tuned hyperparameters from a separate search.

VP and PF stages. Both branches fine-tune from the best TP checkpoint using AdamW with an initial learning rate of 2×10^{-3}, $\mathrm{lr}_f = 0.01$, momentum 0.9, and weight decay 0.025. All scales are trained for 10 epochs. For VP training, only the SAVPE module and cv4 heads are unfrozen; for PF training, only the final classification layers (cv3) are unfrozen.

SEG stage. The segmentation branch also fine-tunes from the best TP checkpoint but uses MuSGD with the same learning rate schedule as the TP stage ($\mathrm{lr}_0 = 1.25 \times 10^{-3}$, $\mathrm{lr}_f = 0.5$). Only the segmentation-specific layers (cv5, proto) are unfrozen, and training runs for 10 epochs across all scales.

Training data. All four stages share the same training data, consisting of three grounding datasets: Objects365-v1 [63], GQA [23], and Flickr30k [56]. Following YOLOE [79], we adopt the YOLOE data engine to refine the annotations of all three datasets. Fig. S3 illustrates representative before-and-after examples, showing that the refined annotations ex-

Setting	YOLO26n	YOLO26s	YOLO26m	YOLO26l	YOLO26x
muon_w	0.45	0.50	0.45	0.45	0.50
sgd_w	0.55	0.50	0.55	0.55	0.60
cls_w	1.00	1.00	1.00	1.00	1.00
o2m	0.10	0.10	0.10	1.00	1.00

Table S4. Checkpoint-recorded internal training parameters associated with the Objects365-v1 pretraining checkpoints.

Setting	YOLO26n	YOLO26s	YOLO26m	YOLO26l	YOLO26x
Optimizer and Schedule					
optimizer	MuSGD	MuSGD	MuSGD	MuSGD	MuSGD
end2end	True	True	True	True	True
imgsz	640	640	640	640	640
batch	128	128	128	128	128
epochs	245	70	80	60	40
lr0	0.0054	0.00038	0.00038	0.00038	0.00038
lrf	0.05	0.88	0.88	0.88	0.88
momentum	0.95	0.95	0.95	0.95	0.95
weight_decay	0.00064	0.00027	0.00027	0.00027	0.00027
warmup_epochs	0.98	0.99	0.99	0.99	0.99
close_mosaic	10	10	10	10	10
Loss Weights					
box	5.63	9.83	9.83	9.83	9.83
cls	0.56	0.65	0.65	0.65	0.65
dfl	9.04	0.96	0.96	0.96	0.96

Table S5. Official optimizer, schedule, and loss settings for the released YOLO26 COCO checkpoints.

hibit fewer missing instances.

S7. Additional YOLOE-26 Benchmarks

This subsection collects the full YOLOE-26 benchmark tables complementary to Sec. 4.6.

Prompt-based Detection. Table S12 reports the full LVIS minival comparison for text-prompted and visual-prompted open-vocabulary detection.

Under text prompting, YOLOE-26 consistently improves over prior YOLOE variants at matched scales, with gains of **+3.3/+2.4/+2.6 AP** over YOLOE-11 and **+2.9/+2.8/+1.9 AP** over YOLOE-v8 at the s/m/l scales. YOLOE-26x achieves the best text-prompted result among the compared methods, reaching **40.6 AP**. At the lightweight end, YOLOE-26n reaches 24.7 AP with only 3.9M parameters, indicating that the gains also extend to resource-constrained settings. For text-prompted inference, the end-to-end head remains close to the Non-E2E variant, trailing by at most 1.1 AP across scales. Under visual prompting, the gap ranges from 1.0 to 2.6 AP. YOLOE-26 again improves consistently over earlier YOLOE families, and YOLOE-26x attains the best visual-prompted result at **38.5 AP**. In addition, the s/m/l models obtain higher AP_r than their text-prompted counterparts, suggesting that visual prompts are particularly beneficial for rare categories at these scales.

Zero-shot Segmentation. Table S13 reports the full zero-shot segmentation comparison on the LVIS val set.

We extend YOLOE-26 with a lightweight segmentation head and evaluate zero-shot mask prediction on the LVIS val set. YOLOE-26 consistently outperforms prior YOLOE variants across all model scales. Notably, YOLOE-26s achieves 20.5 / 19.1 AP^m (T / V), surpassing YOLOE-v8m (20.8 / 20.3) at less than half the computational cost, demonstrating the efficiency advantage brought by the YOLO26 backbone. At the large scale, YOLOE-26l reaches 24.8 / 24.3 AP^m, exceeding YOLOE-v8l by +1.3 / +2.3 points while also outperforming fine-tuned baselines such as YOLOWorldv2-L† (19.8) by a significant margin. Scaling further to YOLOE-26x yields the best overall performance of 27.4 / 26.7 AP^m, with particularly strong gains on rare categories (AP^m_r = 24.9 / 23.3).

Setting	YOLO26n	YOLO26s	YOLO26m	YOLO26l	YOLO26x
mosaic	0.91	0.99	0.99	0.99	0.99
mixup	0.01	0.05	0.43	0.43	0.43
copy_paste	0.08	0.40	0.30	0.40	0.40
scale	0.56	0.90	0.95	0.95	0.95
fliplr	0.61	0.30	0.30	0.30	0.30
degrees	1.11	~ 0	~ 0	~ 0	~ 0
shear	1.46	~ 0	~ 0	~ 0	~ 0
translate	0.07	0.27	0.27	0.27	0.27
hsv_h	0.01	0.01	0.01	0.01	0.01
hsv_s	0.64	0.35	0.35	0.35	0.35
hsv_v	0.57	0.19	0.19	0.19	0.19
bgr	0.11	0.00	0.00	0.00	0.00

Table S6. Official size-specific augmentation settings for the released YOLO26 COCO checkpoints.

Setting	YOLO26n	YOLO26s	YOLO26m	YOLO26l	YOLO26x
muon_w	0.53	0.44	0.44	0.44	0.44
sgd_w	0.67	0.48	0.48	0.48	0.48
cls_w	2.74	3.48	3.48	3.48	3.48
o2m	1.00	0.71	0.71	0.71	0.71
topk	8	5	5	5	5

Table S7. Checkpoint-recorded internal training parameters associated with the released YOLO26 COCO checkpoints.

Prompt-free Detection. Table S14 reports the full prompt-free open-vocabulary detection comparison on LVIS `minival`.

YOLOE-26 achieves competitive prompt-free detection across the model family. At the standard Non-E2E operating point, YOLOE-26x attains the best AP in Table S14 with **31.1 AP**. YOLOE-26l reaches 28.0 AP with only 23.6M parameters, comparable to GenerateU-L [38] (27.9 AP, 467M parameters) while using nearly 20$\times$ fewer parameters. Relative to YOLOE-v8 and YOLOE-11, YOLOE-26s/m/l (Non-E2E) improve AP by **0.8–1.7** and **0.9–2.0**, respectively. The E2E head remains competitive, trailing the Non-E2E variant by only **0.7–1.2 AP** while removing post-processing.

Model	Params (M)	GFLOPs	Latency (ms)	AP^{val}	AP^{val}_{50}	AP^{val}_{75}	AP^{val}_{S}	AP^{val}_{M}	AP^{val}_{L}
S-scale									
YOLOv9-S [81]	**7**	27	3.5	46.8	61.8	48.6	25.7	49.9	61.0
YOLOv10-S [78]	**7**	22	**2.5**	46.3	63.0	50.4	26.8	51.0	63.8
YOLOv12-S [65]	9	21	2.6	48.0	65.0	51.8	29.8	53.2	65.6
YOLO11-S [70]	9	22	**2.5**	47.0	63.9	50.7	29.0	51.7	64.4
D-FINE-S [55]	10	25	3.5	48.5	65.6	52.6	29.1	52.2	65.4
DEIM-S [21]	10	25	3.5	**49.0**	**65.9**	**53.1**	**30.4**	52.6	65.7
YOLO26s	10	21	**2.5**	48.6	65.8	52.8	29.5	**53.2**	**65.8**
M-scale									
YOLOv9-M [81]	20	77	6.4	51.4	67.2	54.6	32.0	55.7	66.4
YOLOv10-M [78]	**15**	59	4.7	51.1	68.1	55.8	33.8	56.5	67.0
RT-DETRv2-S [46]	20	60	**4.6**	48.1	65.1	57.4	36.1	**57.9**	**70.8**
YOLOv12-M [65]	20	68	4.9	52.5	69.6	57.1	35.7	58.2	68.8
YOLO11-M [70]	20	68	4.7	51.5	68.5	55.7	33.4	57.1	67.9
D-FINE-M [55]	19	**57**	5.6	52.3	69.8	56.4	33.2	56.5	70.2
DEIM-M [21]	19	**57**	5.6	52.7	70.0	57.3	35.3	56.7	69.5
YOLO26m	20	68	4.7	**53.1**	**70.7**	**57.7**	**36.7**	57.8	68.9
L-scale									
YOLOv9-C [81]	26	103	7.2	53.0	70.2	57.8	36.2	58.5	69.3
YOLOv10-L [78]	**24**	120	7.3	53.2	70.1	58.1	35.8	58.5	69.4
YOLOv12-L [65]	26	89	6.8	53.7	70.7	58.5	36.9	59.5	69.9
YOLO11-L [70]	25	87	**6.2**	53.4	70.1	58.2	35.6	59.1	69.2
D-FINE-L [55]	31	91	8.1	54.0	71.6	58.4	36.5	58.0	71.9
DEIM-L [21]	31	91	8.1	54.7	72.4	59.4	36.9	**59.6**	71.8
RT-DETRv2-M [46]	31	92	6.9	49.9	67.5	58.6	35.8	58.6	**72.1**
YOLO26l	25	86	**6.2**	**55.0**	**72.5**	**60.0**	**38.4**	59.5	71.1
X-scale									
YOLOv9-E [81]	58	193	16.8	55.6	72.8	60.6	40.2	61.0	71.4
YOLOv10-X [78]	**30**	**160**	**10.7**	54.4	71.3	59.3	37.0	59.8	70.9
YOLOv12-X [65]	59	199	11.8	55.2	72.0	60.2	39.6	60.7	70.9
YOLO11-X [70]	57	195	11.3	54.7	71.6	59.5	37.7	59.7	70.2
D-FINE-X [55]	62	202	12.9	55.8	73.7	60.2	37.3	60.5	73.4
DEIM-X [21]	62	202	12.9	56.5	74.0	61.5	38.8	61.4	**74.2**
RT-DETRv2-X [46]	76	259	13.9	54.3	72.8	58.8	35.8	58.8	72.1
YOLO26x	56	194	11.8	**57.5**	**75.0**	**62.7**	**41.8**	**62.1**	73.3

Table S8. **Comparison with recent real-time detectors on COCO `val2017`, shown in grouped s/m/l/x form for readability.** Literature baselines are taken from the corresponding primary papers or release benchmarks. To keep the computational-complexity columns aligned with released benchmark tables, we report only the standard YOLO26 NMS variants here. For YOLO26, Params/GFLOPs/T4 TensorRT latency are taken from Table 7, while the AP breakdowns come from the corresponding NMS validation runs.

Model	Size (px)	mAP^{box} val 50–95	mAP^{mask} val 50–95	CPU ONNX (ms)	T4 TRT10 (ms)	Params (M)	FLOPs (B)
YOLO11n-seg	640	38.9	32.0	65.9	1.8	2.9	9.7
YOLO26n-seg (E2E)	640	39.6	33.9	53.5	2.1	2.7	9.1
YOLO26n-seg (Non-E2E)	640	40.6	34.4	52.7	2.0	2.7	9.1
YOLO11s-seg	640	46.6	37.8	117.6	2.9	10.1	33.0
YOLO26s-seg (E2E)	640	47.3	40.0	118.4	3.3	10.4	34.2
YOLO26s-seg (Non-E2E)	640	48.2	40.5	102.4	3.2	10.4	34.2
YOLO11m-seg	640	51.5	41.5	281.6	6.3	22.4	113.2
YOLO26m-seg (E2E)	640	52.5	44.1	328.2	6.7	23.6	121.5
YOLO26m-seg (Non-E2E)	640	53.1	44.4	337.7	6.9	23.6	121.5
YOLO11l-seg	640	53.4	42.9	344.2	7.8	27.6	132.2
YOLO26l-seg (E2E)	640	54.4	45.5	387.0	8.0	28.0	139.8
YOLO26l-seg (Non-E2E)	640	55.2	46.0	395.8	8.2	28.0	139.8
YOLO11x-seg	640	54.7	43.8	664.5	15.8	62.1	296.4
YOLO26x-seg (E2E)	640	56.5	47.0	787.0	16.4	62.8	313.5
YOLO26x-seg (Non-E2E)	640	57.2	47.5	795.9	16.8	62.8	313.5

Table S9. Comparisons of instance segmentation models on COCO validation set, where ‘E2E‘ corresponds to using the one-to-one branch for inference without NMS and ‘Non-E2E‘ corresponds to using the one-to-many branch for inference with NMS. CPU ONNX and T4 TensorRT latency values exclude NMS time.

Model	Size (px)	mAP val 50–95	AP val 50	CPU ONNX (ms)	T4 TRT10 (ms)	Params (M)	FLOPs (B)
YOLO11n-pose	640	50.0	81.0	52.4	1.7	2.9	7.4
YOLO26n-pose (E2E)	640	57.2	83.3	40.3	1.8	2.9	7.5
YOLO26n-pose (Non-E2E)	640	57.0	83.1	40.8	1.8	2.9	7.5
YOLO11s-pose	640	58.9	86.3	90.5	2.6	9.9	23.1
YOLO26s-pose (E2E)	640	63.0	86.6	85.3	2.7	10.4	23.9
YOLO26s-pose (Non-E2E)	640	62.9	86.6	88.6	2.8	10.4	23.9
YOLO11m-pose	640	64.9	89.4	187.3	4.9	20.9	71.4
YOLO26m-pose (E2E)	640	68.8	89.9	218.0	5.0	21.5	73.1
YOLO26m-pose (Non-E2E)	640	68.8	89.6	228.9	5.1	21.5	73.1
YOLO11l-pose	640	66.1	89.9	247.7	6.4	26.1	90.3
YOLO26l-pose (E2E)	640	70.4	90.5	275.4	6.5	25.9	91.3
YOLO26l-pose (Non-E2E)	640	70.3	90.6	268.9	6.5	25.9	91.3
YOLO11x-pose	640	69.5	91.1	488.0	12.1	58.8	202.8
YOLO26x-pose (E2E)	640	71.6	91.6	565.4	12.2	57.6	201.7
YOLO26x-pose (Non-E2E)	640	71.7	91.0	574.4	12.4	57.6	201.7

Table S10. Comparisons of pose estimation models on COCO validation set, where ‘E2E‘ corresponds to using the one-to-one branch for inference without NMS and ‘Non-E2E‘ corresponds to using the one-to-many branch for inference with NMS. CPU ONNX and T4 TensorRT latency values exclude NMS time.

Model	**Size (px)**	**mAP val 50–95**	**AP val 50**	**AP val 75**	**CPU ONNX (ms)**	**T4 TRT10 (ms)**	**Params (M)**	**FLOPs (B)**
YOLO11n-obb	1024	49.7	78.4	52.1	117.6	4.4	2.7	16.8
YOLO26n-obb	1024	52.4	78.9	56.8	97.7	2.8	2.5	14.0
YOLO11s-obb	1024	51.4	79.5	54.4	219.4	5.1	9.7	57.1
YOLO26s-obb	1024	54.8	80.9	60.4	218.0	4.9	9.8	55.1
YOLO11m-obb	1024	52.8	80.9	56.1	562.8	10.1	20.9	182.8
YOLO26m-obb	1024	55.3	81.0	60.7	579.2	10.2	21.2	183.3
YOLO11l-obb	1024	52.9	81.0	56.6	712.5	13.5	26.1	231.2
YOLO26l-obb	1024	56.2	81.6	62.2	735.6	13.0	25.6	230.0
YOLO11x-obb	1024	54.1	81.3	57.8	1408.6	28.6	58.8	519.1
YOLO26x-obb	1024	56.7	81.7	62.6	1485.7	30.5	57.6	516.5

Table S11. Comparisons of OBB models on the DOTA-v1.0 test set, where YOLO26 models are evaluated using the one-to-one branch without NMS.

Model	Size	Prompt	$mAP^{e2e}_{50–95}$	$mAP_{50–95}$	mAP_r	mAP_c	mAP_f	Params (M)	FLOPs (B)
GLIP-T [34]	–	T	—	26.0	20.8	21.4	31.0	232	—
GLIPv2-T [96]	–	T	—	29.0	—	—	—	232	—
GDINO-T [43]	–	T	—	27.4	18.1	23.3	32.7	172	—
DetCLIP-T [91]	–	T	—	34.4	26.9	33.9	36.3	155	—
G-1.5 Edge [61]	–	T	—	33.5	28.0	34.3	33.9	—	—
T-Rex2 [26]	–	V	—	37.4	29.9	33.9	41.8	—	—
YOLOE-26n	640	T/V	23.7 / 20.9	24.7 / 21.9	20.5 / 17.6	24.1 / 22.3	26.1 / 22.4	**3.9 / 3.1**	**6.1**
YOLOWorldv2-S [9]	640	T	—	24.4	17.1	22.5	27.3	13	—
YOLOE-v8s [79]	640	T/V	—	27.9 / 26.2	22.3 / 21.3	27.8 / 27.7	29.0 / 25.7	12.3 / 12.6	29.8
YOLOE-11s [79]	640	T/V	—	27.5 / 26.3	21.4 / 22.5	26.8 / 27.1	29.3 / 26.4	**10.7 / 10.9**	22.7
YOLOE-26s	640	T/V	**29.9 / 27.1**	**30.8 / 28.6**	**23.9 / 25.1**	**29.6 / 27.8**	**33.0 / 29.9**	10.7 / 11.0	**21.9**
YOLOWorldv2-M [9]	640	T	—	32.4	28.4	29.6	35.5	29	—
YOLOE-v8m [79]	640	T/V	—	32.6 / 31.0	26.9 / 27.0	31.9 / 31.7	34.4 / 31.1	26.4 / 28.4	80.7
YOLOE-11m [79]	640	T/V	—	33.0 / 31.4	26.9 / 27.1	32.5 / 31.9	34.5 / 31.7	**21.0 / 24.8**	**70.4**
YOLOE-26m	640	T/V	**35.4 / 31.3**	**35.4 / 33.9**	**31.1 / 33.4**	**34.7 / 34.0**	**36.9 / 33.8**	21.3 / 25.1	70.6
YOLOWorldv2-L [9]	640	T	—	35.5	25.6	34.6	38.1	48	—
YOLOE-v8l [79]	640	T/V	—	35.9 / 34.2	33.2 / 33.2	34.8 / 34.6	37.3 / 34.1	43.5 / 47.3	167.6
YOLOE-11l [79]	640	T/V	—	35.2 / 33.7	29.1 / 28.1	35.0 / 34.6	36.5 / 33.8	**26.0 / 29.8**	**89.5**
YOLOE-26l	640	T/V	**36.8 / 33.7**	**37.8 / 36.3**	**35.1 / 37.6**	**37.6 / 36.2**	**38.5 / 36.1**	25.5 / 29.3	89.0
YOLOE-26x	640	T/V	**39.5 / 36.2**	**40.6 / 38.5**	**37.4 / 35.3**	**40.9 / 38.8**	**41.0 / 38.8**	55.2 / 65.2	197.7

Table S12. Detection results with text and visual prompts on LVIS minival. T = Text prompt, V = Visual prompt; metric values are reported as T / V.

Model	Prompt	AP^m	AP^m_r	AP^m_c	AP^m_f
YOLOE-v8s	T / V	17.7 / 16.8	15.5 / 13.5	16.3 / 16.7	20.3 / 18.2
YOLOE-11s	T / V	17.6 / 17.1	16.1 / 14.4	15.6 / 16.8	20.5 / 18.6
YOLOE-26s	T / V	**20.5 / 19.1**	**18.4 / 16.1**	**18.6 / 18.2**	**23.4 / 21.4**
YOLOE-26n	T / V	15.1 / 13.9	11.9 / 11.0	13.9 / 13.4	17.9 / 15.6
YOLOWorld-M†	T	16.7	12.6	14.6	20.8
YOLOWorldv2-M†	T	17.8	13.9	15.5	22.0
YOLOE-v8m	T / V	20.8 / 20.3	17.2 / 17.0	19.2 / 20.1	24.2 / 22.0
YOLOE-11m	T / V	21.1 / 21.0	17.2 / 18.3	19.6 / 20.6	24.4 / 22.6
YOLOE-26m	T / V	**23.8 / 22.9**	**24.2 / 23.3**	**25.6 / 26.6**	**29.4 / 28.4**
YOLOWorld-L†	T	19.1	14.2	17.2	23.5
YOLOWorldv2-L†	T	19.8	15.0	17.5	23.6
YOLOE-v8l	T / V	23.5 / 22.0	21.9 / 16.5	21.6 / 22.1	26.4 / 24.3
YOLOE-11l	T / V	22.6 / 22.5	19.3 / 20.5	20.9 / 21.7	26.0 / 24.1
YOLOE-26l	T / V	**24.8 / 24.3**	**21.9 / 21.9**	**23.3 / 23.6**	**27.8 / 26.1**
YOLOE-26x	T / V	**27.4 / 26.7**	**24.9 / 23.3**	**26.2 / 26.6**	**29.8 / 28.4**

Table S13. **Segmentation evaluation on LVIS.** We evaluate all models on LVIS `val` set with the standard AP^m reported. YOLOE supports both text (T) and visual cues (V) as inputs. † indicates that the pretrained models are fine-tuned on `LVIS-Base` data for segmentation head. In contrast, we evaluate YOLOE in a zero-shot manner without utilizing any images from LVIS during training.

Model	Params	AP	AP_r	AP_c	AP_f	FLOPs
GenerateU-T [38]	297	26.8	20.0	24.9	29.8	—
GenerateU-L [38]	467	27.9	22.3	25.2	31.4	—
YOLOE-26n	**2.3**	16.6/17.7	15.7/15.8	15.3/16.4	17.9/19.2	**5.3**
YOLOE-v8s [79]	13	21.0	19.1	21.3	21.0	—
YOLOE-11s [79]	**11**	20.6	18.4	20.2	21.3	—
YOLOE-26s	9.0	**21.4/22.6**	**16.2/20.2**	**20.1/20.9**	**23.5/24.5**	20.8
YOLOE-v8m [79]	29	24.7	22.2	24.5	25.3	—
YOLOE-11m [79]	**24**	25.5	21.6	25.5	26.1	—
YOLOE-26m	19.4	**25.7/26.4**	**26.7/24.5**	**24.0/25.0**	**26.9/27.9**	68.4
YOLOE-v8l [79]	47	27.2	23.5	**27.0**	**28.0**	—
YOLOE-11l [79]	**29**	26.3	22.7	25.8	27.5	—
YOLOE-26l	23.6	**27.2/28.0**	**26.3/25.7**	25.7/**26.8**	**28.7/29.5**	86.8
YOLOE-26x	53.1	**29.9/31.1**	**27.5/28.9**	**29.1/30.7**	**31.1/31.7**	194.4

Table S14. Prompt-free detection on LVIS minival. AP is Fixed $AP_{50\text{–}95}$; subscripts denote rare/common/frequent splits. YOLOE-26 reports E2E / Non-E2E. GenerateU-T/L use Swin-T/L backbones. YOLOE-26 trains on Objects365-v1 [63], GQA [23], Flickr30k [56].

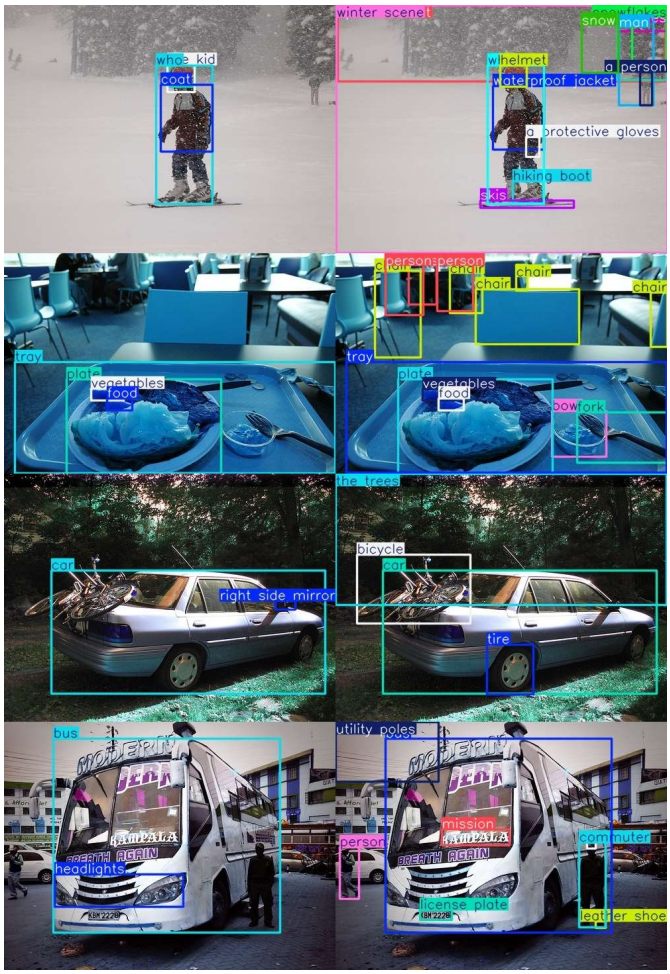


Figure S3. Comparison of four samples before (left) and after (right) refinement by the YOLOE data engine.